



树的构建

- 对于任意图 $G(m, n) = \langle E, V \rangle$ ，可以通过逐步构建树 $G_T = \langle V_T, E_T \rangle$ 的方法，判断 G 是否是树：
 - 1) 对于 $(u_0, v_0) \in E$ ，有 $V_T = \{u_0, v_0\}$, $E_T = \{(u, v)\}$
 - 2) 对于 $(u, v) \in E$
 - 若 $u \in V_T$ 并且 $v \in V_T$ ，则 $E_T \cup \{(u, v)\}$ 有圈
 - 若 $u \in V_T$ 并且 $v \notin V_T$ ，或者 $u \notin V_T$ 并且 $v \in V_T$ ，则 $V_T = V_T \cup \{u, v\}$, $E_T = E_T \cup \{(u, v)\}$, $E = E \setminus \{(u, v)\}$
 - 3) 若 $\text{len}(V) \neq \text{len}(V_T)$ ，则图 G 不连通。



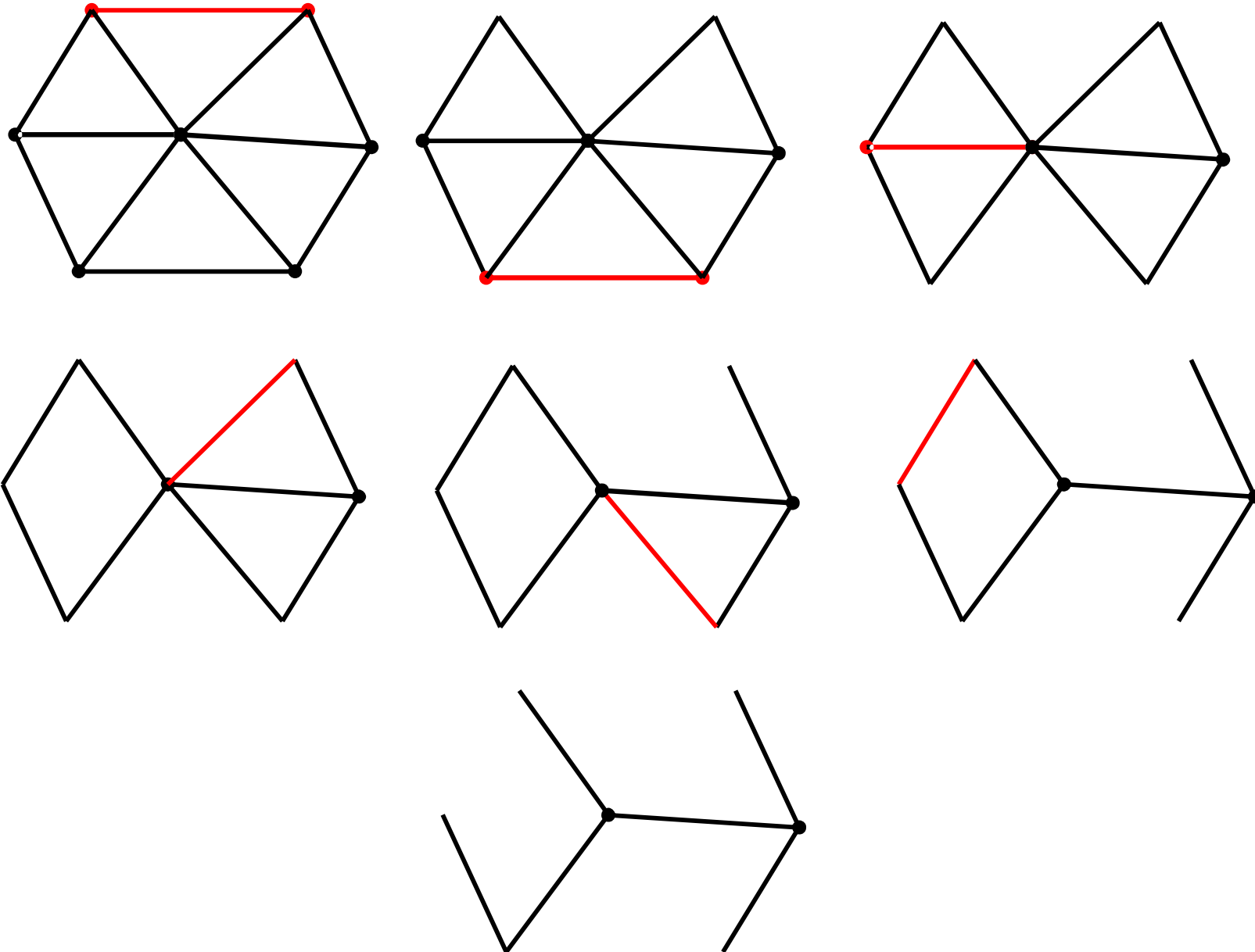
树的构建

- 避圈法
- 初值
 - $V_t = \{u, v\}$
 - $E_t = \{(u, v)\}$
- 当 $\text{len}(T) \leq (n - 1)$
 - 避圈增加一条边

```
def graph2tree(V,E):
    E=sorted(E)
    (u,v)=E[0]
    Vt={u,v}
    Et={(u,v)}
    E=set(E)
    n=len(V)
    while(len(Et) < (n-1)):
        m=len(Vt)
        for (u,v) in E:
            if((u in Vt and v not in Vt)
               or (u not in Vt and v in Vt)):
                Vt=Vt | {u,v}
                Et=Et | {(u,v)}
        E=E-Et
        if(m == len(Vt)):
            break
    return [Vt,Et]
```



树的构建



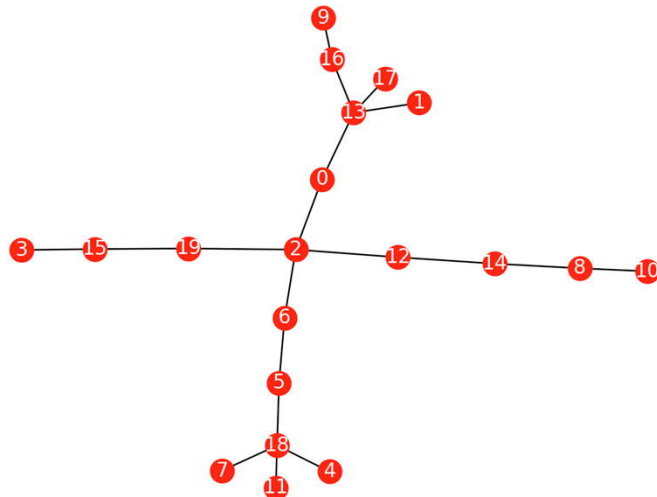


树的判断

- $G(n, m)$
- 连通并且 $m = n - 1$
- 如何判断连通?

```
import graph.graph as gg
[V,E]=gg.creategraph(20,40)
gg.drawgraph(E)
[Vt,Et]=gg.graph2tree(V,E)
gg.drawgraph(Et)
print(istree(Vt,Et))
```

True



```
def istree(V,E):
    tv=True
    E=sorted(E)
    (u,v)=E[0]
    Vt={u,v}
    E=set(E)-{(u,v)}
    Et={(u,v)}
    while(E != set({})):
        for (u,v) in E:
            if (u in Vt) and (v in Vt):
                tv=False
                break
            if ((u in Vt) and (v not in Vt)) or ((u
not in Vt) and (v in Vt)):
                Vt=Vt | {u,v}
                E=E-{(u,v)}
                Et=Et|{(u,v)}
        if(tv == False):
            break
    if((len(Vt)-1) != len(Et)):
        tv=False
    return tv
```

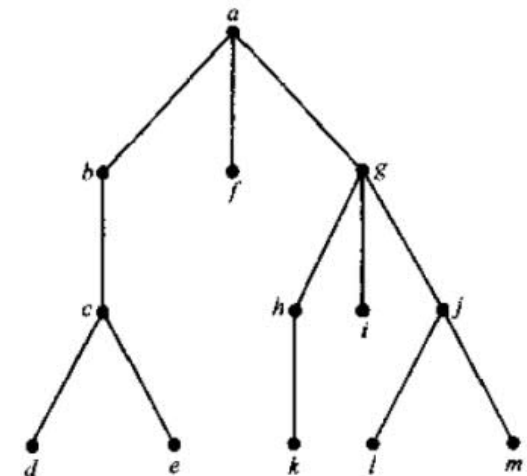


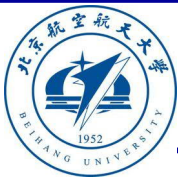
根树

- **定义 12.2.2** 设 $G = \langle V, E \rangle$ 是有向图，存在**唯一节点**称为**根节点** u_r ， $d_i(u_r) = 0$ ，即 u_r 的入度为 0，对于任意其它节点 v ， $d_i(v) = 1$ ，即 v 的入度为 1，并使得 $u_r \Rightarrow v$ ，称 G 为根树，记为 $T = \langle V, E \rangle$ 。
- $\forall u \forall v (u \in V \wedge v \in V \wedge u \neq v \rightarrow u \Rightarrow v) \wedge \forall u (u \in V \rightarrow \neg(u \Rightarrow u)) \wedge \exists \neg u_r (d_i(u_r) = 0 \wedge \forall v (v \in V \wedge v \neq u_r \rightarrow d_i(v) = 1))$

- 例：设 $G = \langle V, E \rangle$ 是根树，节点 a 为根，其中

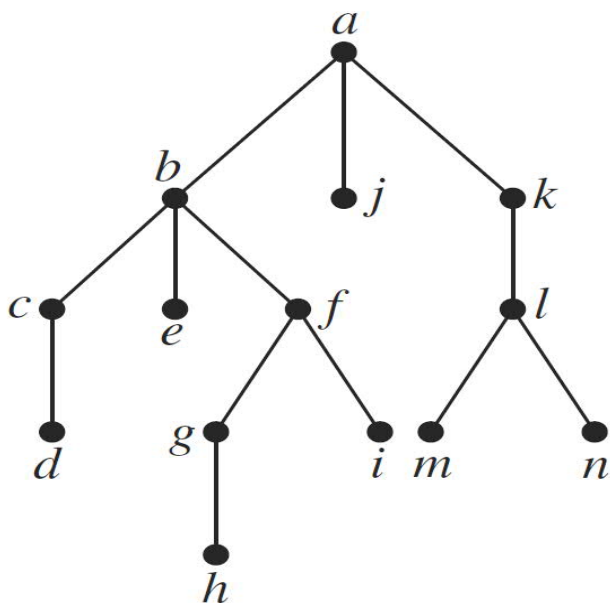
$V = \{ 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm' \}$
 $E = \{ ('a', 'b'), ('a', 'f'), ('a', 'g'), ('b', 'c'), ('c', 'd'), ('c', 'e'), ('g', 'h'), ('g', 'i'), ('g', 'j'), ('h', 'k'), ('j', 'l'), ('j', 'm') \}$





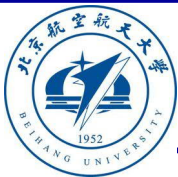
有序树

- **定义** 设 $T = \langle V, E \rangle$ 是根树，并且每个节点的子节点是有序的，则称为有序树。
- $\langle u, v, k \rangle$ 表示 u 的第 k 个子节点 v 。



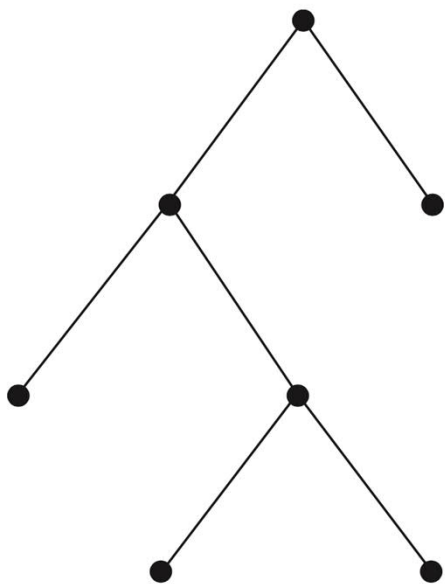
```
def main( ):
    global a,b,c,d,e,f,g,h,i,j,k,l,m,n

V={'a','b','c','d','e','f','g','h','i','j','k','l','m','n'}
n=['n',[],[],[]]
m=['m',[],[],[]]
l=['l',m,[],n]
k=['k',[],l,[]]
j=['j',[],[],[]]
i=['i',[],[],[]]
h=['h',[],[],[]]
g=['g',[],h,[]]
f=['f',g,[],i]
e=['e',[],[],[]]
d=['d',[],[],[]]
c=['c',[],d,[]]
b=['b',c,e,f]
a=['a',b,j,k]
return a
```

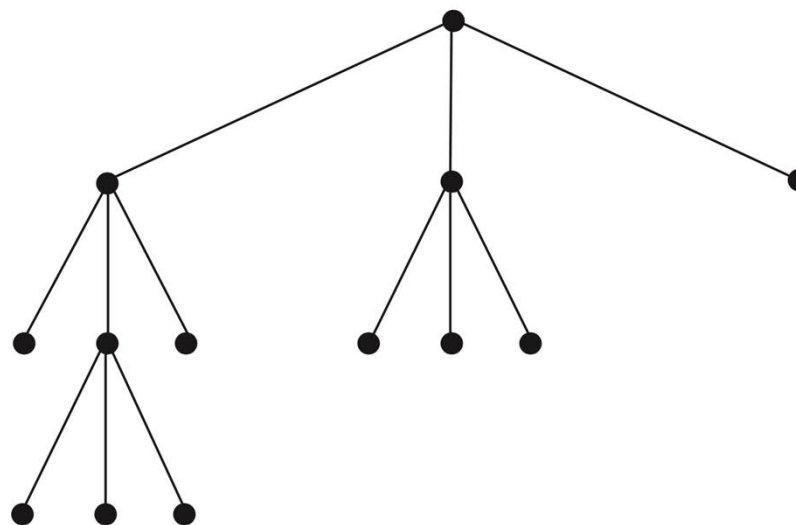


m 元树

- **定义 12.2.3** 设 V 是非空节点集合, $E = \{\langle u, v, k \rangle | u \in V \wedge v \in V, k = 1, 2, \dots, m\}$, $T = \langle V, E \rangle$ 是树, 并且每个节点的最多有 m 个子节点, 则称为 m 元树。
- 如图所示, G_1 为二元树, G_2 为三元树



G_1

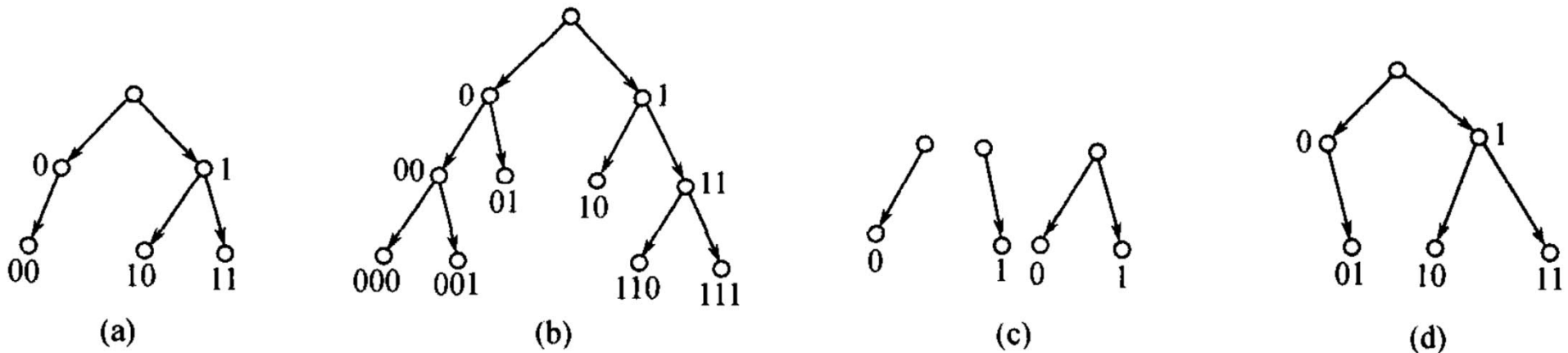


G_2

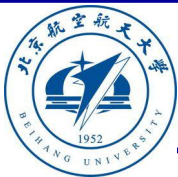


m 元树

- **完全 m 元树**：若每一个节点的引出次数都等于 m 或 0 ；
- **位置 m 元树**：任何节点的 m 个子节点都有固定的位置；
- **二叉树**： $m = 2$ ，则称为二元树（二叉树）；

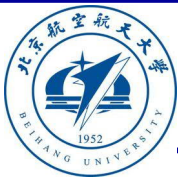


- (a) 是二元树，(b) 是完全二元树；
- (c) 是位置二元树中一个节点的子节点可能的排列；
- (a) 和 (d) 是不同的位置二元树；



本章内容

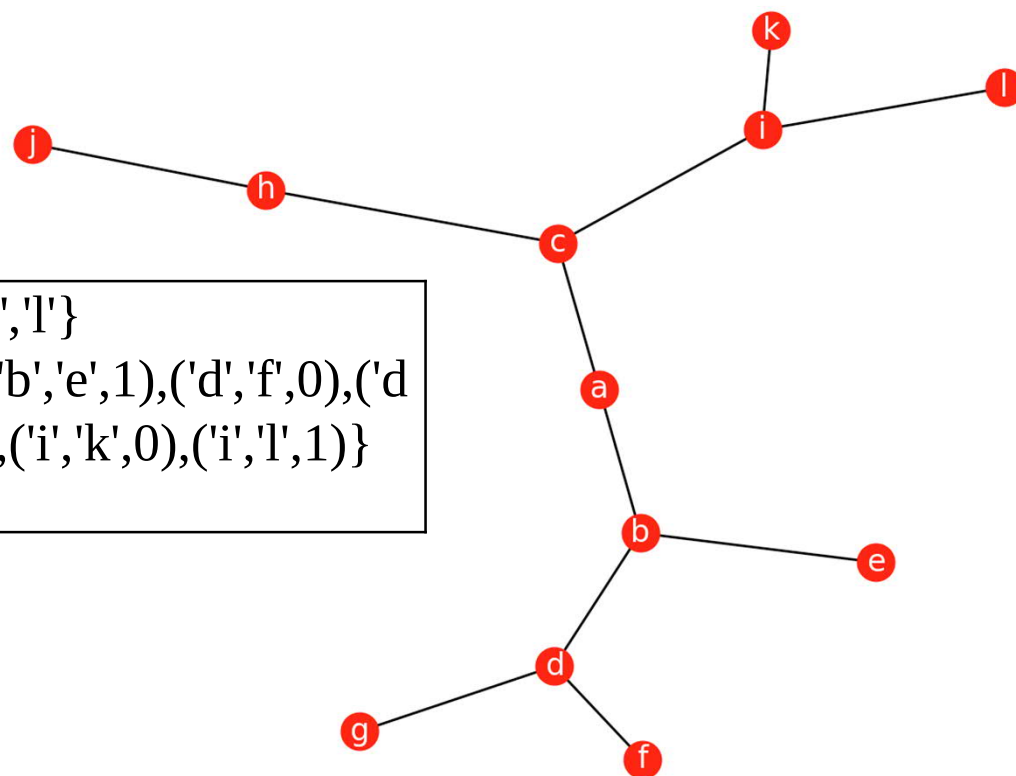
- 树的概念
 - 定义与判断
 - 根树与有序树
- 二叉树及遍历
 - 二叉树的表示
 - 二叉树的遍历
- 霍夫曼编码
 - 霍夫曼树构建
 - 霍夫曼编码
- 最小生成树
 - 最小生成树的定义
 - 普里姆算法
 - 克鲁斯卡尔算法



二叉树的表示

- **定义 12.2.2** 设 $T = \langle V, E \rangle$ 是有序树，并且每个节点最多有2个子节点，则称为有序二叉树，简称二叉树。
- $\forall v(v \in V \wedge v \neq u_r \rightarrow u_r \Rightarrow v) \wedge \forall v(v \in V \wedge v \neq u_r \rightarrow d_I(v) = 1 \wedge d_O(v) \leq 2)$

$V = \{ 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l' \}$
 $E = \{ ('a', 'b', 0), ('a', 'c', 1), ('b', 'd', 0), ('b', 'e', 1), ('d', 'f', 0), ('d', 'g', 1), ('c', 'h', 0), ('c', 'i', 1), ('h', 'j', 0), ('i', 'k', 0), ('i', 'l', 1) \}$





二叉树的表示

- 有序二叉树区分左子树和右子树
- 有序二叉树用表结构形式描述如下：
 - $[]$ 是二叉树；
 - 若 $a \in V$ ，则 $[a]$ 是二叉树；
 - 若 $a \in V$ ， L 和 R 是二叉树，则 $[a, L, R]$ 是二叉树；



二叉树的表示

- 构造有序二叉树的方法如下，其中nodes为节点集
 - 若 $len(nodes) == 0$ ，则return[];
 - 若 $len(nodes) == 1$ ，则[nodes[0]];
 - 在 $0 \sim len(nodes) - 1$ 之间任取 k ，有
 - ① 节点为 $a = nodes.pop(k)$;
 - ② 左子树节点集为 $Lnodes = nodes[0:k]$, 左子树为 $L = creatbitree(Lnodes)$;
 - ③ 右子树节点集为 $Rnodes = nodes[k + 1:]$, 右子树为 $R = creatbitree(Rnodes)$;
 - ④ Return $[a, L, R]$



递归定义函数及实现

<pre>factor(n)=n*factor(n-1) factor(1)=1</pre>	<pre>def factor(n): if(n==1): return 1 return n*factor(n-1)</pre>
<pre>Hanoi(n)=2*Hanoi(n-1)+1 Hanoi(1)=1</pre>	<pre>def Hanoi(n): if(n ==1): return 1 return 2*Hanoi(n-1)+1</pre>
<pre>Fibonacci(n) = Fibonacci(n-1)+Fibonacci(n-2) Fibonacci(1)=1 Fibonacci(2)=1</pre>	<pre>def Fibonacci(n): if(n==1 or n==2): return 1 return Fibonacci(n-1)+Fibonacci(n-2)</pre>
<pre>Ackemann(m,n)= Ackemann(m-1, Ackemann(m,n-1)) Ackemann(0,n)=n+1 Ackemann(m,0)=Ackemann(m-1,1)</pre>	<pre>def Ackemann(m,n): if(m==0): return (n+1) if(n==0): return Ackemann(m-1,1) return Ackemann(m-1, Ackemann(m,n-1))</pre>



二叉树构建算法

■ 二叉树递归定义（set,list结构）

```
def createbitree(nodes):
    nodes=set(nodes)
    if(len(nodes) == 0):
        return [ ]
    if(len(nodes) == 1):
        return list(nodes)
    [a]=random.sample(nodes,1)
    nodes=nodes-{a}
    k=random.randint(0,len(nodes)-1)
    Lnodes=random.sample(nodes,k)
    Lnodes=set(Lnodes)
    Rnodes=nodes-Lnodes
    L=createbitree(Lnodes)
    R=createbitree(Rnodes)
    return [a,L,R]
```

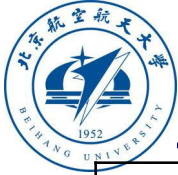
```
def bitree(nodes):
    nodes=list(nodes)
    if(len(nodes) == 0):
        return [ ]
    if(len(nodes) == 1):
        return nodes
    k=int(len(nodes)/2)
    a=nodes.pop(k)
    Lnodes=nodes[0:k]
    Rnodes=nodes[k:]
    L=bitree(Lnodes)
    R=bitree(Rnodes)
    return [a,L,R]
```



二叉树与图

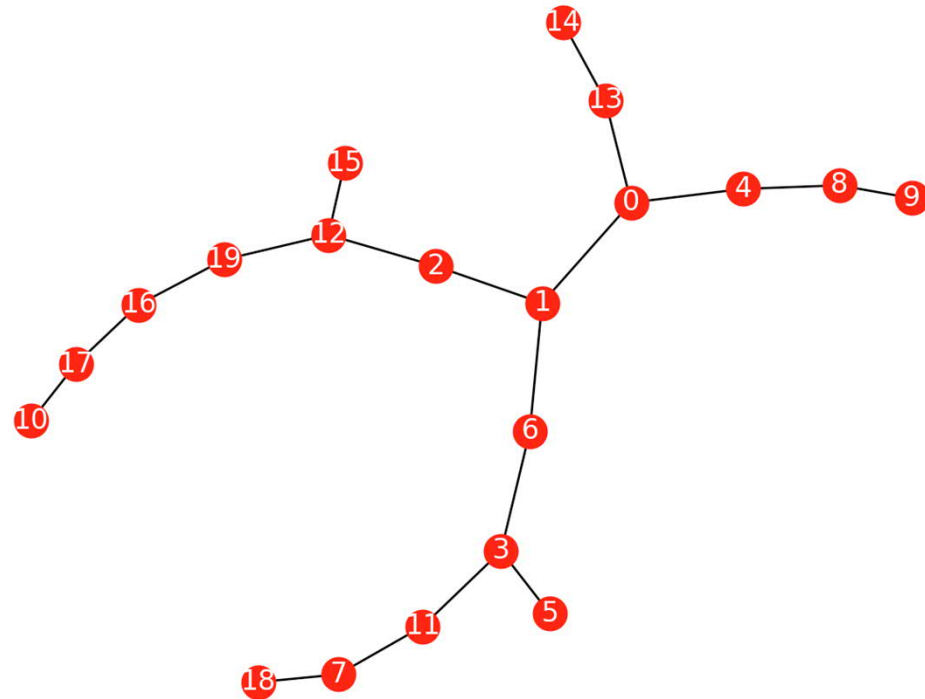
- 从树结构变换为图结构
- (1) 若 $\text{len}(\text{tree}) = 0$ 或 $\text{len}(\text{tree}) = 1$, 则为空集。
- (2) tree有左子树, 则
 $\text{L} = \text{tree2graph}(\text{tree}[1])$
 $\text{gtree} = \text{gtree} \mid \{(\text{tree}[0], \text{tree}[1][0])\} \mid \text{L}$
- (3) tree有右子树, 则
 $\text{R} = \text{tree2graph}(\text{tree}[2])$
 $\text{gtree} = \text{gtree} \mid \{(\text{tree}[0], \text{tree}[2][0])\} \mid \text{R}$

```
def tree2graph(tree):  
    gtree=set({})  
    if len(tree) == 0 | len(tree) == 1:  
        return gtree  
    if len(tree[1]) != 0:  
        L=tree2graph(tree[1])  
        gtree=gtree | {(tree[0],tree[1][0])} | L  
    if len(tree[2]) != 0:  
        R=tree2graph(tree[2])  
        gtree=gtree | {(tree[0],tree[2][0])} | R  
    return gtree
```



二叉树与图

```
import graph.graph as
gg
m = 20
V = range(m)
V = sorted(V)
tree
=gg.createbitree(V)
gtree =
gg.tree2graph(tree)
gg.drawgraph(gtree)
print("tree",tree)
print(gtree)
```



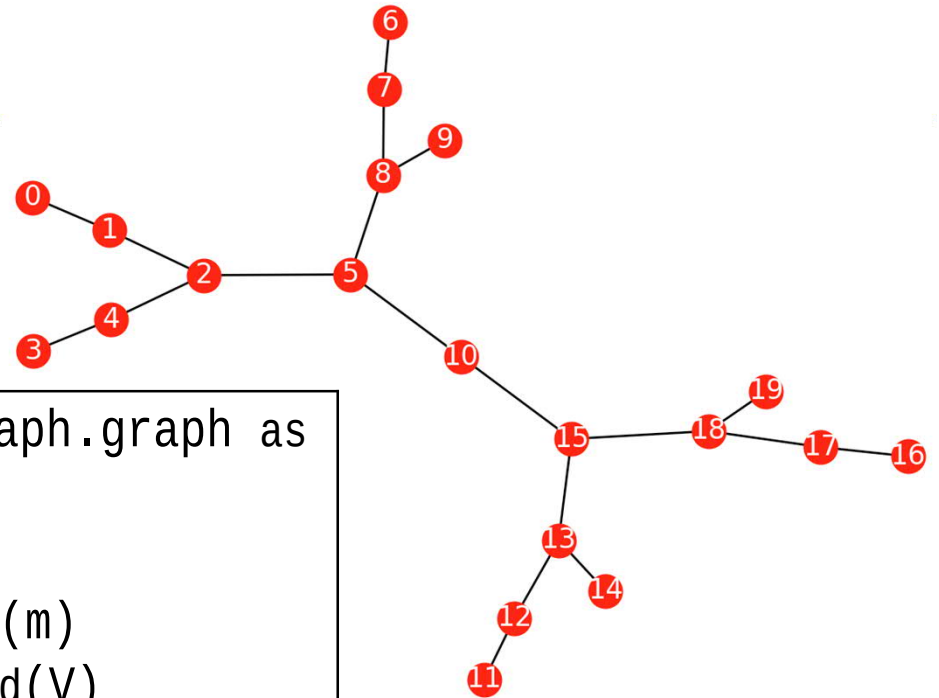
```
tree [2, [12, [19, [], [16, [], [17, [], [10]]]], [15]], [1, [0, [4, [],
[8, [], [9]]], [13, [], [14]]], [6, [], [3, [5], [11, [], [7, [],
[18]]]]]]]
{(11, 7), (0, 4), (2, 1), (0, 13), (12, 19), (8, 9), (1, 0), (1, 6), (7,
18), (3, 5), (3, 11), (13, 14), (17, 10), (19, 16), (2, 12), (4, 8),
(12, 15), (6, 3), (16, 17)}
```



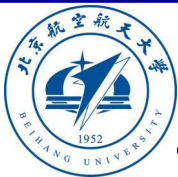
二叉树与图

```
def bitree(nodes):
    nodes =
    list(nodes)
    if (len(nodes)
== 0):
        return []
    if (len(nodes)
== 1):
        return nodes
    k =
    int(len(nodes) / 2)
    a = nodes.pop(k)
    Lnodes =
    nodes[0:k]
```

```
import graph.graph as
gg
m = 20
V = range(m)
V = sorted(V)
tree =gg.bitree(V)
gtree =
gg.tree2graph(tree)
gg.drawgraph(gtree)
print("tree",tree)
print(gtree)
```



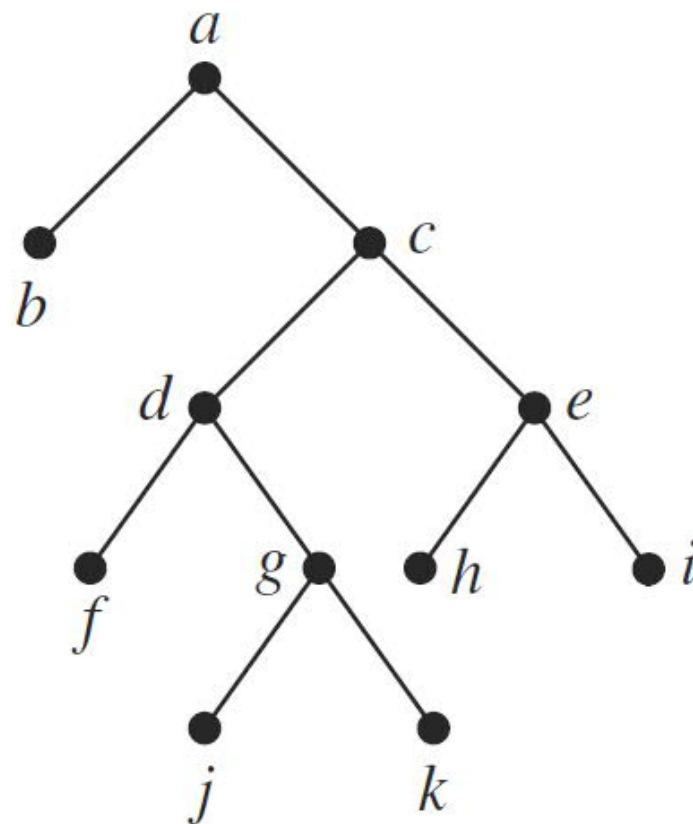
```
tree [10, [5, [2, [1, [0], []], [4, [3], []]], [8, [7, [6], []],
[9]]], [15, [13, [12, [11], []], [14]], [18, [17, [16], []], [19]]]]
{(10, 5), (2, 4), (18, 17), (2, 1), (4, 3), (15, 13), (8, 9), (7, 6),
(1, 0), (12, 11), (5, 2), (10, 15), (13, 14), (5, 8), (8, 7), (17,
16), (18, 19), (13, 12), (15, 18)}
```



二叉树的遍历

- 定义12.3.2 设 $T = \langle E, V \rangle$ 是二叉树，访问 T 的每个节点的过程称为遍历。
- 二叉树子树结构
 - $[d_k, T_1, T_2]$

```
def main( ):
    g=['g',['j'],['k']]
    e=['e',['h'], ['i']]
    d=['d',['f'],g]
    c=['c',d,e]
    a=['a', ['b'],c]
    return a
```

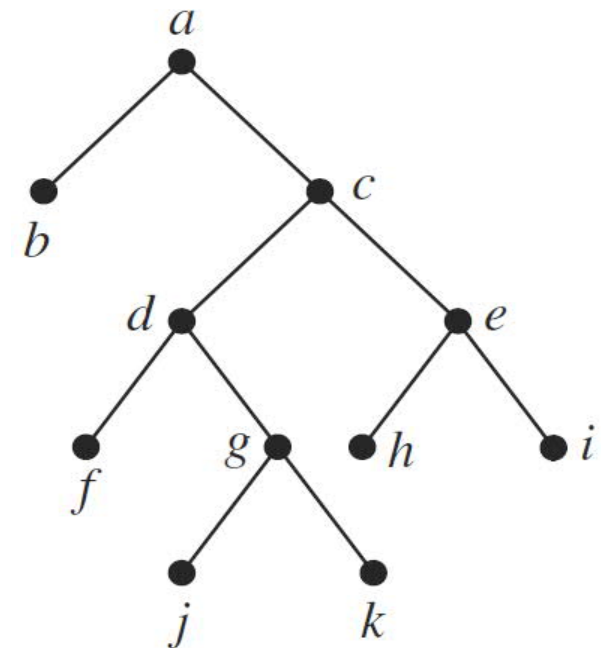




前序遍历

- 设有二叉树 T ，前序遍历是指：首先访问树根节点，然后访问左子树 T_1 ，最后访问右子树 T_2 。
 - 若 $tree == []$ ，则 $traversallist = []$
 - 若 $tree == [u]$ ，则 $traversallist = [u]$
 - 否则 $traversallist = [u] + T_1 + T_2$

- ['a', 'b', 'c', 'd', 'f', 'g', 'j', 'k', 'e', 'h', 'i']

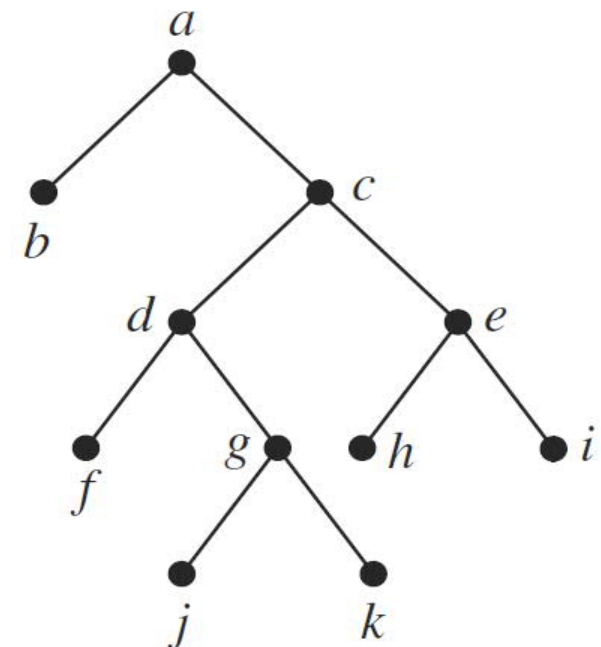




中序遍历

- 设有二叉树 T ，中序遍历是指：首先访问左子树 T_1 ，然后访问树根节点，最后访问右子树 T_2 。
 - 若 $tree == []$ ，则 $traversallist = []$
 - 若 $tree == [u]$ ，则 $traversallist = [u]$
 - 否则 $traversallist = T_1 + [u] + T_2$

- ['b', 'a', 'f', 'd', 'j', 'g', 'k', 'c', 'h', 'e', 'i']

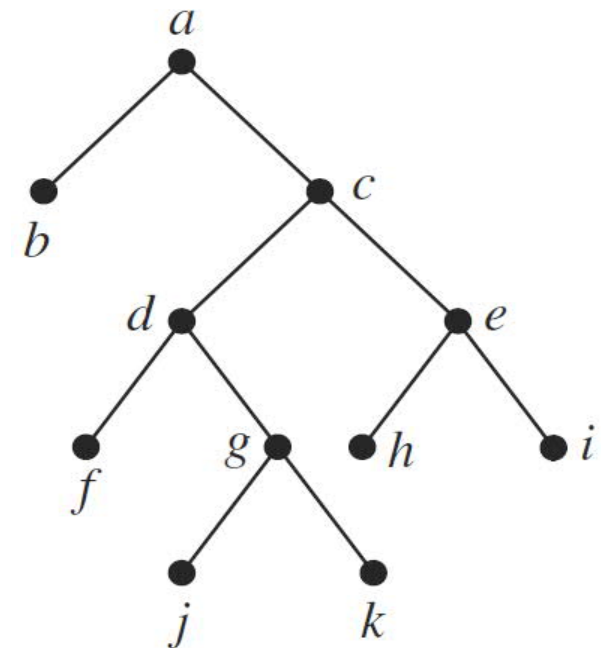




后序遍历

- 设有二叉树 T ，中序遍历是指：首先访问左子树 T_1 ，然后访问右子树 T_2 ，最后访问树根节点。
 - 若 $tree == []$ ，则 $traversallist = []$
 - 若 $tree == [u]$ ，则 $traversallist = [u]$
 - 否则 $traversallist = T_1 + T_2 + [u]$

- ['b', 'f', 'j', 'k', 'g',
'd', 'h', 'i', 'e', 'c', 'a']



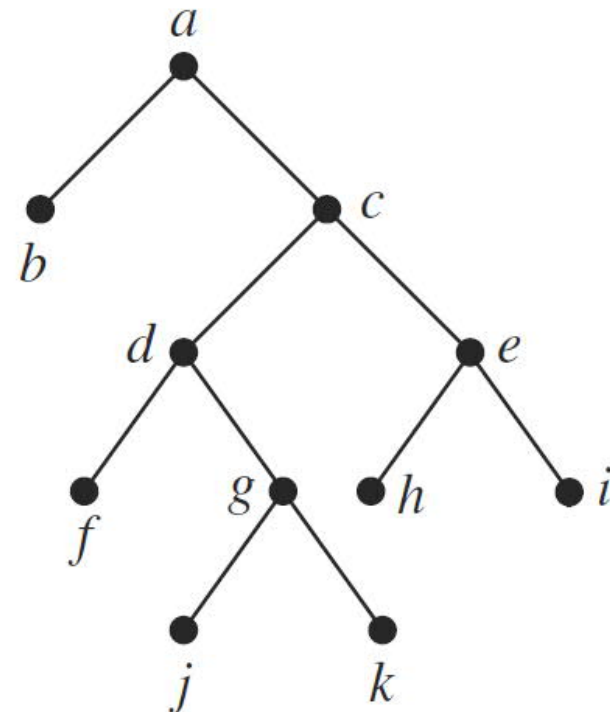


前序遍历算法

```
def preordertraversal(tree):  
    if(len(tree)==0):  
        return []  
    if(len(tree)==1):  
        return [tree[0]]  
    else:  
        return [tree[0]]+preordertraversal(tree[1])+preordertraversal(tree[2])
```

```
import graph.graph as gg  
tree=['a', ['b'], ['c', ['d', ['f'], ['g', ['j'], ['k']],  
    ['e', ['h'], ['i']]]]  
print(gg.preordertraversal(tree))
```

`['a', 'b', 'c', 'd', 'f', 'g', 'j', 'k', 'e', 'h', 'i']`



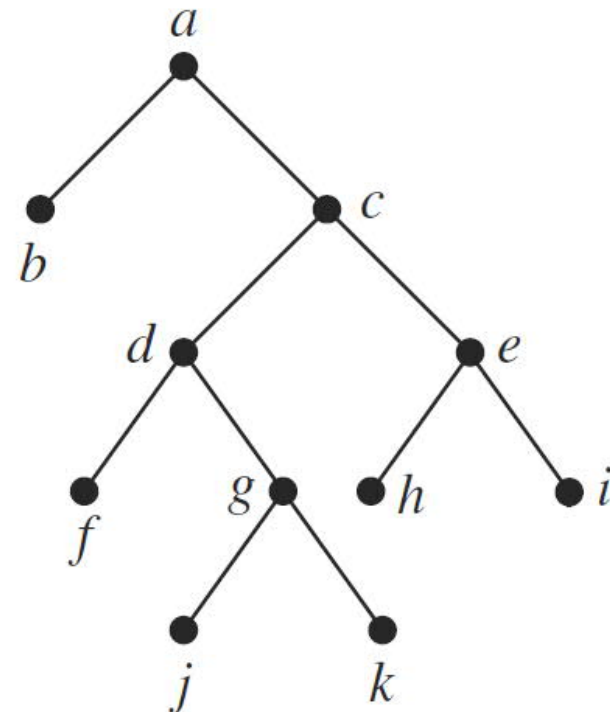


中序遍历算法

```
def inordertraversal(tree):  
    if(len(tree)==0):  
        return []  
    if(len(tree)==1):  
        return [tree[0]]  
    else:  
        return preordertraversal(tree[1])+[tree[0]]+preordertraversal(tree[2])
```

```
import graph.graph as gg  
tree=['a', ['b'], ['c', ['d', ['f'], ['g', ['j'], ['k']],  
['e', ['h'], ['i']]]]]  
print(gg.inordertraversal(tree))
```

`['b', 'a', 'f', 'd', 'j', 'g', 'k', 'c', 'h', 'e', 'i']`



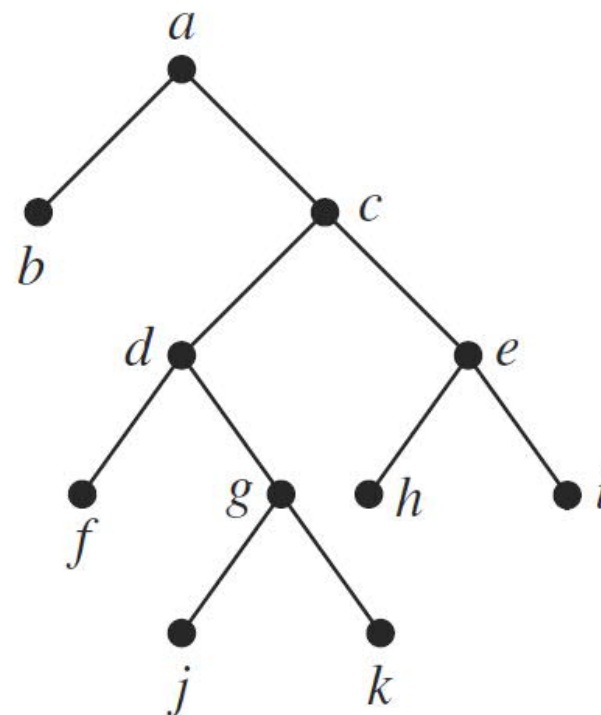


后序遍历算法

```
def postordertraversal(tree):  
    if(len(tree)==0):  
        return []  
    if(len(tree)==1):  
        return [tree[0]]  
    else:  
        return preordertraversal(tree[1])+preordertraversal(tree[2])+[tree[0]]
```

```
import graph.graph as gg  
tree=['a', ['b'], ['c', ['d', ['f'], ['g', ['j'], ['k']],  
['e', ['h'], ['i']]]]]  
print(gg.postordertraversal(tree))
```

`['b', 'f', 'j', 'k', 'g', 'd', 'h', 'i', 'e', 'c', 'a']`





二叉树遍历

■ 三种二叉树遍历演示

```
import graph.graph as gg
tree
=['a',['b'],['c',['d',['f'],['g',['j'],['k']]],['e',['h'],['i']]]]
preorderlist = gg.preordertraversal(tree)
inorderlist = gg.inordertraversal(tree)
postorderlist = gg.postordertraversal(tree)
print("前序遍历",preorderlist)
print("中序遍历",inorderlist)
print("后序遍历",postorderlist)
```

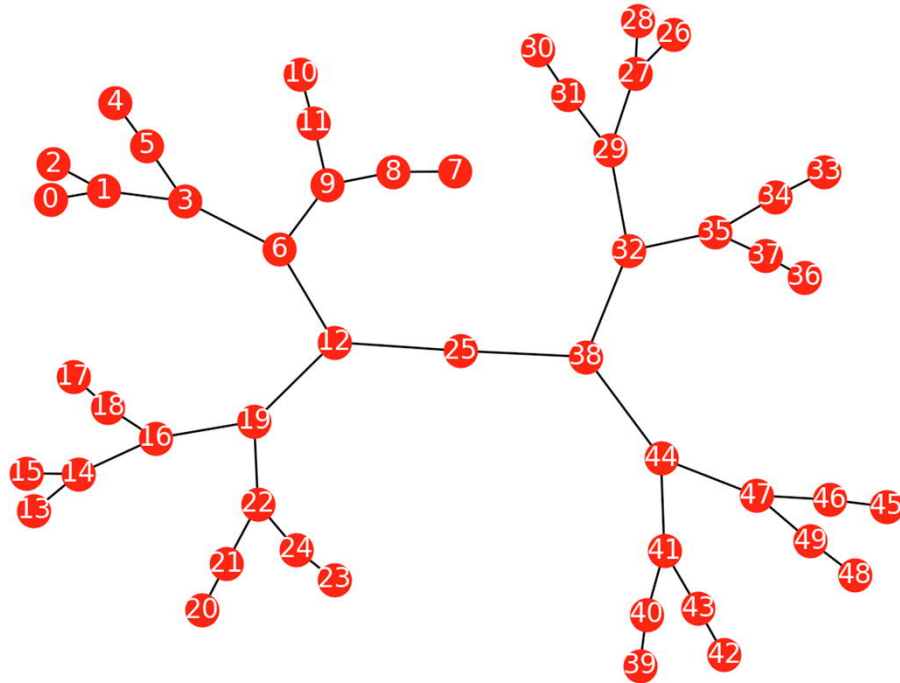
前序遍历 ['a', 'b', 'c', 'd', 'f', 'g', 'j', 'k', 'e', 'h', 'i']

中序遍历 ['b', 'a', 'f', 'd', 'j', 'g', 'k', 'c', 'h', 'e', 'i']

后序遍历 ['b', 'f', 'j', 'k', 'g', 'd', 'h', 'i', 'e', 'c', 'a']



二叉树遍历

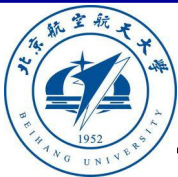


```
import graph.graph as gg
m = 50
V = range(m)
V = set(V)
tree = gg.bitree(V)
gtree = gg.tree2graph(tree)
gg.drawgraph(gtree)
print("tree", tree)
preorderlist =
gg.preordertraversal(tree)
inorderlist =
gg.inordertraversal(tree)
postorderlist =
gg.postordertraversal(tree)
print("前序遍历", preorderlist)
```

```
tree [25, [12, [6, [3, [1, [0], [2]], [5, [4], []], [9, [8, [7], []], [11, [10], []]]],
[19, [16, [14, [13], [15]], [18, [17], []], [22, [21, [20], []], [24, [23], []]]]], [38,
[32, [29, [27, [26], [28]], [31, [30], []], [35, [34, [33], []], [37, [36], []]]], [44,
[41, [40, [39], []], [43, [42], []], [47, [46, [45], []], [49, [48], []]]]]]
```

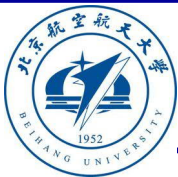
```
前序遍历 [25, 12, 6, 3, 1, 0, 2, 5, 4, 9, 8, 7, 11, 10, 19, 16, 14, 13, 15, 18, 17, 22,
21, 20, 24, 23, 38, 32, 29, 27, 26, 28, 31, 30, 35, 34, 33, 37, 36, 44, 41, 40, 39, 43,
42, 47, 46, 45, 49, 48]
```

```
中序遍历 [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21,
22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43,
44, 45, 46, 47, 48, 49]
```



本章内容

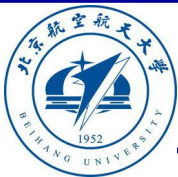
- 树的概念
 - 定义与判断
 - 根树与有序树
- 二叉树及遍历
 - 二叉树的表示
 - 二叉树的遍历
- 霍夫曼编码
 - 霍夫曼树构建
 - 霍夫曼编码
- 最小生成树
 - 最小生成树的定义
 - 普里姆算法
 - 克鲁斯卡尔算法



问题提出

- 通信的编码解码问题
- 等长编码：ASCII编码
- 不等长编码：
 - 无前缀编码
- 每个字出现概率不同

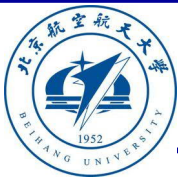
符号	频率	编码1	长度1	期望长度1	编码2	长度2	期望长度2
A	30%						
B	20%						
C	15%						
D	10%						
E	10%						
F	6%						
G	5%						
H	4%						



问题提出

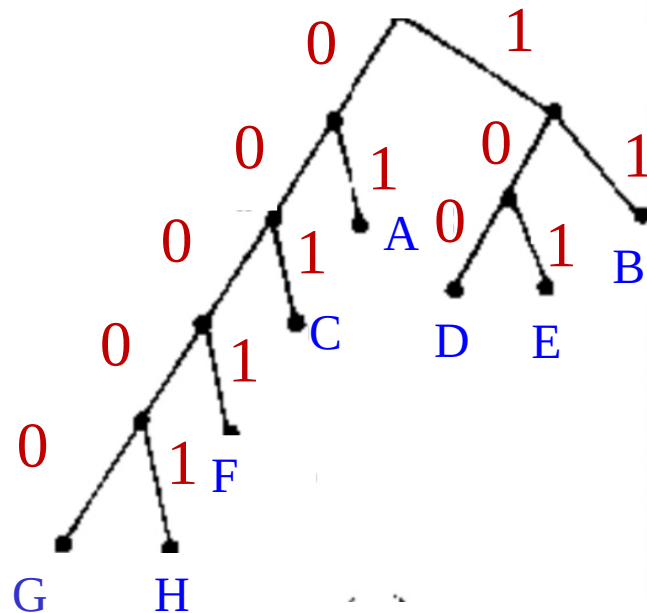
- 通信的编码解码问题
- 等长编码：ASCII编码
- 不等长编码：
 - 无前缀编码
- 每个字出现概率不同

符号	频率	编码1	长度1	期望长度1	编码2	长度2	期望长度2
A	30%	000	3	0.9			
B	20%	001	3	0.6			
C	15%	010	3	0.45			
D	10%	011	3	0.3			
E	10%	100	3	0.3			
F	6%	101	3	0.18			
G	5%	110	3	0.15			
H	4%	111	3	0.12			

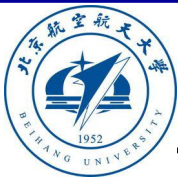


问题提出

- 通信的编码解码问题
- 等长编码：ASCII编码
- 不等长编码：
 - 无前缀编码
- 每个字出现概率不同

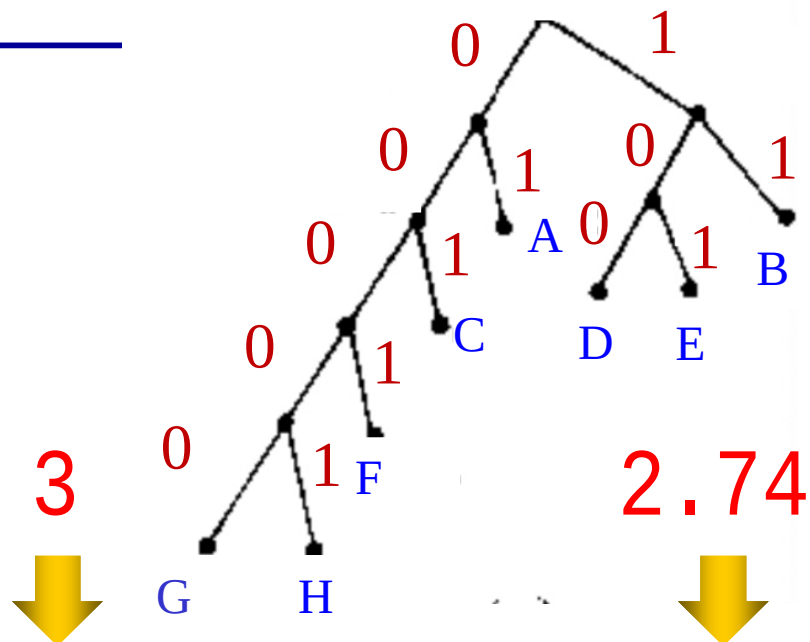


符号	频率	编码1	长度1	期望长度1	编码2	长度2	期望长度2
A	30%	000	3	0.9			
B	20%	001	3	0.6			
C	15%	010	3	0.45			
D	10%	011	3	0.3			
E	10%	100	3	0.3			
F	6%	101	3	0.18			
G	5%	110	3	0.15			
H	4%	111	3	0.12			



问题提出

- 通信的编码解码问题
- 等长编码：ASCII编码
- 不等长编码：
 - 无前缀编码
- 每个字出现概率不同



符号	频率	编码1	长度1	期望长度1	编码2	长度2	期望长度2
A	30%	000	3	0.9	01	2	0.6
B	20%	001	3	0.6	11	2	0.4
C	15%	010	3	0.45	001	3	0.45
D	10%	011	3	0.3	100	3	0.3
E	10%	100	3	0.3	101	3	0.3
F	6%	101	3	0.18	0001	4	0.24
G	5%	110	3	0.15	00000	5	0.25
H	4%	111	3	0.12	00001	5	0.2



霍夫曼编码熵

目标:

- 用较短（长）的序列去表示出现频率高（低）的字母，

问题转化为:

- 求出叶的权分别为0.30, 0.20, 0.15, 0.10, 0.10, 0.06, 0.05, 0.04的最优二叉树，然后用这样的二叉树产生前缀编码传输上述给定的字母。

■ Huffman编码

- 1) 给定字母集 $C = \{c_1, c_2, \dots, c_n\}$ 及频率 $f(c_1), \dots, f(c_n)$;
- 2) 设有 n 个叶结点，分别以 $f(c_1), \dots, f(c_n)$ 为权;
- 3) 在所有入度为0的结点，选出两个权最小的结点 v, v' ，添加一个新的分支节点 u ，使得 u 以 v 和 v' 为儿子结点，且 $f(u) = f(v) + f(v')$;
- 4) 重复3)直至只有一个入度为0的结点。



Huffman算法

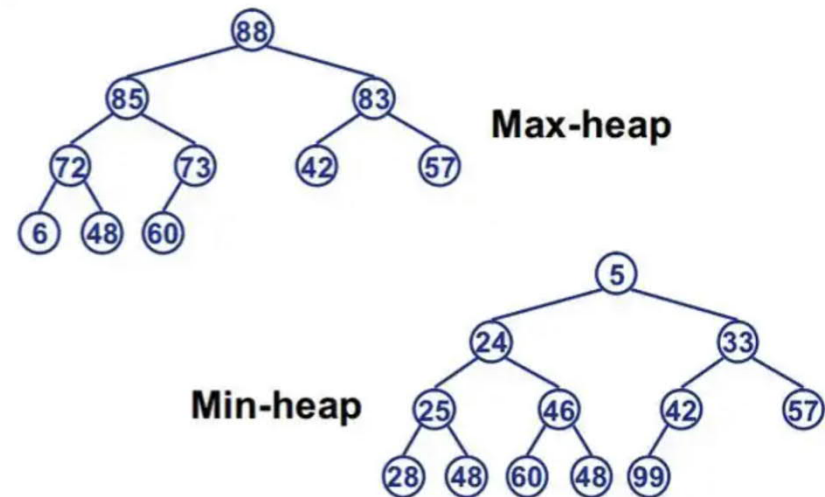
- 建立Huffman树的主要运算是插入和删除最小频度字符，所以用最小堆。

算法HUFFMAN

输入： n 个字符的集合 $C = \{c_1, c_2, \dots, c_n\}$ 及频率 $\{f(c_1), \dots, f(c_1)\}$ 。

输出： C 的Huffman树 (V, T) 。

1. 根据频度将所有字符插入最小堆 H
2. $V \leftarrow C$; $T = \{\}$
3. for $j \leftarrow 1$ to $n-1$
4. $c \leftarrow \text{DELETMIN}(H)$
5. $c' \leftarrow \text{DELETMIN}(H)$
6. $f(v) \leftarrow f(c) + f(c')$ //新节点 v
7. $\text{INSERT}(H, v)$
8. $V = V \cup \{v\}$
9. $T = T \cup \{(v, c), (v, c')\}$ // c, c' 为 T 中 v 的孩子
10. end for





Huffman算法

例：在计算机通信中要传输A, B, C, D, E, F, G, H八个字母，他们出现频率为A:30%, B:20%, C:15%, D:10%, E:10%, F:6%, G:5%, H:4%。给出一个最佳编码，使得通讯中出现的二进制数字尽可能少。

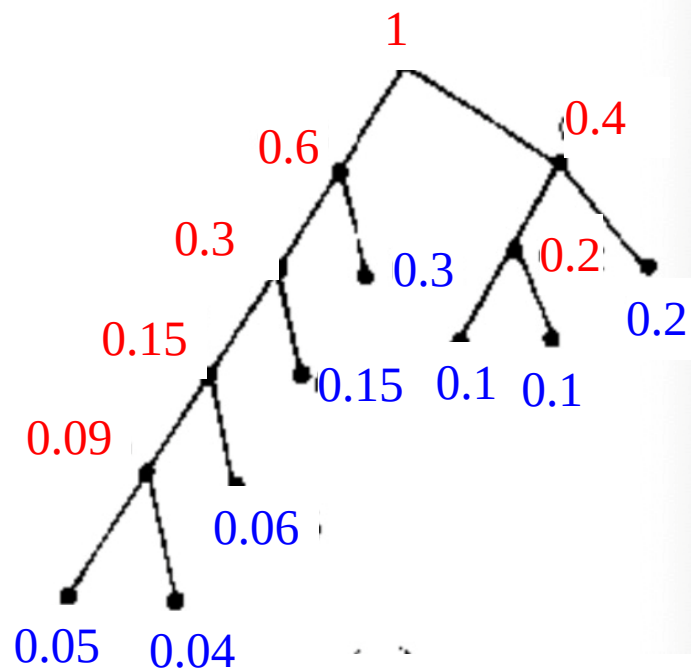
问题转化为：

- 求出叶的权分别为0.04, 0.05, 0.06, 0.1, 0.1, 0.15, 0.20, 0.3 的最优二叉树，然后用这样的二叉树产生前缀编码传输上述给定的字母。

0.04	0.05	0.06	0.1	0.1	0.15	0.20	0.3
	0.09	0.06	0.1	0.1	0.15	0.20	0.3
		0.15	0.1	0.1	0.15	0.20	0.3
		0.15		0.2	0.15	0.20	0.3
			0.2	0.3	0.20		0.3
				0.3	0.40		0.3
					0.40		0.6
							1.0

最佳编码

A: 01, B: 11, C: 001, D:100, E:101,
F: 0001, G: 00000, H:00001





熵

- $W=[['A',0.08], ['B',0.1], ['C',0.12], ['D',0.15], ['E',0.2], ['F',0.35]]$
- $C=[['A', '111'], ['B', '110'], ['E', '10'], ['C', '011'], ['D', '010'], ['F', '00']]$
- 熵= $\sum_k P_k \log_2 P_k$
- 概率平均码长= $\sum_k P_k M_k$

<pre>import math W=[['a',0.08], ['b',0.1], ['c',0.12], ['d',0.15], ['e',0.2], ['f',0.35]] r = 0 for [w,u] in W: r=r-w*math.log2(w) print(r)</pre>	<pre>W=[['a',0.08], ['b',0.1], ['c',0.12], ['d',0.15], ['e',0.2], ['f',0.35]] C=[['a', '111'], ['b', '110'], ['c', '011'], ['d', '010'], ['e', '10'], ['f', '00']] l=0 for k in range(len(C)): [u,w]=C[k] [p,v]=W[k] l=l+p*len(w) print(l)</pre>
2.3957996159499615	2.45



霍夫曼编码熵

```
def codingentropy(W, C):
    W0 = W
    W = []
    for [w, a] in W0:
        W = W + [[a, w]]
    r0 = 0
    r = 0
    W = sorted(W)
    C = sorted(C)
    for k in range(len(W)):
        r0 -= W[k][1] *
math.log2(W[k][1])
        r += W[k][1] *
len(C[k][1])
        r0 = math.floor((r0) * 1000)
/ 1000
        r = math.floor((r) * 1000)
/ 1000
    return [r0, r]
```

```
import graph.graph as gg
```

```
W=[[0.08,'a'], [0.1,'b'], [0.12,'c'],
[0.15,'d'], [0.2,'e'], [0.35,'f']]
C=[['a','111'], ['b','110'], ['c','011
'], ['d','010'], ['e','10'], ['f','00']]
[r, l] = gg.codingentropy(W, C)
print(r, l)
```

2.395

2.45



生成符号频率

- 通过frequencylist(), 统计/随机生成字符的频率

```
def frequencylist(V):  
    #  
    V=['a','b','c','d','e','f','g','h','i','j','k','l','m','n',  
    'o','p','q','r','s','t','u','v','w','x','y','z']  
    W0=[]  
    s=0  
    for u in V:  
        w=random.randint(0,1000)  
        W0=W0+[[u,w]]  
        s=s+w  
    W=[]  
    for [u,w] in W0:  
        w=math.floor((w/s)*10000)/10000  
        W=W+[[u,w]]  
    return W
```

```
import graph.graph as gg  
V=['a','b','c','d','e','f','g','h','i','j','k','l','m','n',  
n,'o','p','q','r','s','t','u','v','w','x','y','z']  
W=gg.frequencylist(V)  
print(W)
```

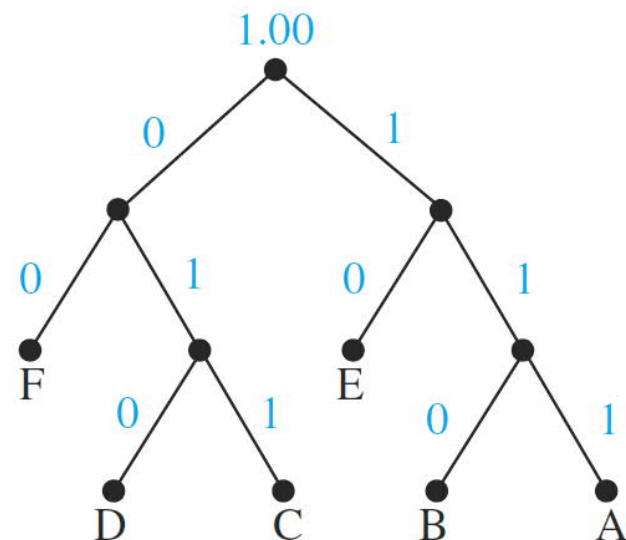
```
[[0.0479, 'a'], [0.0069, 'b'], [0.0689, 'c'],  
[0.0335, 'd'], [0.0218, 'e'], [0.022, 'f'],  
[0.0647, 'g'], [0.048, 'h'], [0.0547, 'i'],  
[0.0216, 'j'], [0.0415, 'k'], [0.0422, 'l'],  
[0.0302, 'm'], [0.0314, 'n'], [0.0324, 'o'],  
[0.0078, 'p'], [0.0371, 'q'], [0.0156, 'r'],  
[0.0245, 's'], [0.0378, 't'], [0.0559, 'u'],  
[0.0414, 'v'], [0.0545, 'w'], [0.049, 'x'],  
[0.0576, 'y'], [0.05, 'z']]
```



霍夫曼树构建与编码

■ 霍夫曼树构建

- 选择最小权值元素 $[w_0, t_0], [w_1, t_1]$
- 生成新元素 $[[w_0 + w_1, [w_0, t_0], [w_1, t_1]]]$
- 重排序
 - $[[1.0,$
 - $[0.38, [0.18, [0.08, 'A'], [0.1, 'B']], [0.2, 'E']],$
 - $[0.62, [0.27, [0.12, 'C'], [0.15, 'D']], [0.35, 'F']]]]$



■ 搜索霍夫曼树，每个从根节点到叶节点路径的01序列，是叶节点符号的编码

$C = [['A', '111'], ['B', '110'], ['E', '10'], ['C', '011'], ['D', '010'], ['F', '00']]$



霍夫曼树构建

- 利用程序构建霍夫曼树的思路
 - $tree = W$
 - $tree = sorted(tree)$
 - $tree = [[tree[0][0] + tree[1][0], tree[0], tree[1]]] + tree[2:]$



霍夫曼树

```
def Huffmantree(W):
```

```
    W0=[]
```

```
    for [a,w] in W:
```

```
        W0=W0+[[w,a]]
```

```
    tree=sorted(W0)
```

```
    while len(tree) != 1:
```

```
        w=tree[0][0]+tree[1][0]
```

```
        w=math.floor(w*10000)/10000
```

```
        tree=[[w,tree[0],tree[1]]]+tree[2:]
```

```
        tree=sorted(tree)
```

```
    return tree[0]
```

```
import graph.graph as gg
```

```
W = [[0.08,'a'], [0.1,'b'], [0.12,'c'], [0.15,'d'], [0.2,'e'], [0.35,'f']]
```

```
tree=gg.Huffmantree(W)
```

```
print(tree)
```

```
[1.0, [0.38, [0.18, [0.08, 'a'], [0.1, 'b']], [0.2, 'e']], [0.62, [0.27, [0.12, 'c'],  
[0.15, 'd']], [0.35, 'f']]]
```



霍夫曼树与图

```
def Huffmantree2graph(tree):
    gtree=set({})
    if len(tree) == 2:
        return gtree
    L=Huffmantree2graph(tree[1])
    R=Huffmantree2graph(tree[2])
    a=math.floor(tree[0]*10000)/10000
    l=math.floor(tree[1][0]*10000)/10000
    r=math.floor(tree[2][0]*10000)/10000
    gtree= {(a,l)} | {(a,r)}|L|R
    return gtree
```

```
def htree2graph(G):
    V=set({})
    for [u,v] in G:
        V=V|{u,v}
    V=sorted(V)
    V=V[::-1]
    E=set({})
    for [u,v] in G:
        E=E|{(V.index(u),V.index(v))}
    return [V,E]
```

```
import graph.graph as gg
W = [[0.08,'a'], [0.1,'b'], [0.12,'c'],
[0.15,'d'], [0.2,'e'], [0.35,'f']]
tree=gg.Huffmantree(W)
G = gg.Huffmantree2graph(tree)
[V,E]=gg.htree2graph(G)
print("G:",G,"V:",V,"E:",E)
```

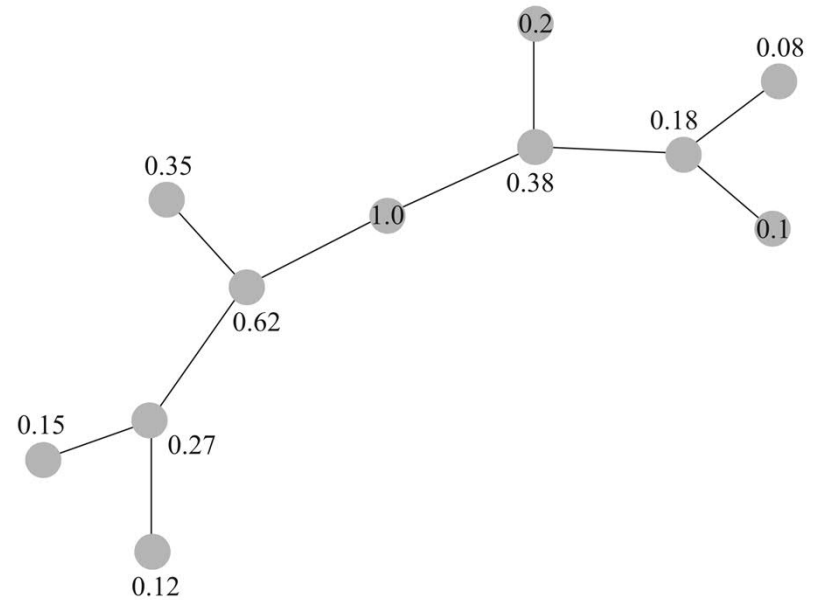
```
G: {(0.38, 0.2), (0.38, 0.18), (1.0, 0.62),
(0.27, 0.12), (1.0, 0.38), (0.18, 0.08), (0.18,
0.1), (0.27, 0.15), (0.62, 0.35), (0.62, 0.27)}
V: [1.0, 0.62, 0.38, 0.35, 0.27, 0.2, 0.18,
0.15, 0.12, 0.1, 0.08]
E: {(0, 1), (1, 3), (1, 4), (0, 2), (2, 6), (6, 10), (4, 8), (2, 5), (6, 9), (4, 7)}
```



霍夫曼树与图

```
import graph.graph as gg

W = [[0.08, 'a'], [0.1, 'b'],
      [0.12, 'c'], [0.15, 'd'],
      [0.2, 'e'], [0.35, 'f']]
tree = gg.Huffmantree(W)
print(tree)
gtree=gg.Huffmantree2graph(tree)
gg.drawgraph(gtree)
print(gtree)
```



```
[1.0, [0.38, [0.18, [0.08, 'a'], [0.1, 'b']], [0.2, 'e']], [0.62, [0.27, [0.12, 'c'], [0.15, 'd']], [0.35, 'f']]]
{(0.38, 0.2), (0.38, 0.18), (1.0, 0.62), (0.27, 0.12), (1.0, 0.38), (0.18, 0.08), (0.18, 0.1), (0.27, 0.15), (0.62, 0.35), (0.62, 0.27)}
```



霍夫曼编码

- 霍夫曼编码是一种基于霍夫曼树的最优编码方式。
huffmancoding(tree)求根节点的左子树霍夫曼编码和右子树霍夫曼编码，并且求树tree的霍夫曼编码。
- huffmancoding(subtree,code)求子树霍夫曼编码
 - 若len(subtree)==2, subtree是叶节点，则
[subtree[1],code]
 - 否则，左子树编码为code+ '0'和右子树编码为code+ '1'，分别递归求霍夫曼编码



霍夫曼编码

```
def huffmancoding(subtree,code):  
    if len(subtree) == 2:  
        return [[subtree[1],code]]  
    else:  
        node0=subtree[1]  
        node1=subtree[2]  
        code=huffmancoding(node0,code+'0')+huffmancoding(node1,code+'1')  
    return code
```

```
def HuffmanCoding(tree):  
    code0=huffmancoding(tree[1],'0')  
    code1=huffmancoding(tree[2],'1')  
    return code0+code1
```

```
import graph.graph as gg  
W = [[0.08,'a'], [0.1,'b'], [0.12,'c'], [0.15,'d'], [0.2,'e'], [0.35,'f']]  
tree = gg.Huffmantree(W)  
C=gg.HuffmanCoding(tree)  
print(C)
```

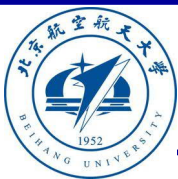
```
[['a', '000'], ['b', '001'], ['e', '01'], ['c', '100'], ['d', '101'], ['f', '11']]
```



霍夫曼树与编码

```
import graph.graph as gg
V = ['a','b','c','d','e','f','g','h','i','j','k','l','m','n','o','p','q','r','s','t','u','v','w','x','y','z']
W = gg.frequencylist(V)
tree = gg.Huffmantree(W)
C = gg.HuffmanCoding(tree)
[r,l] = gg.codingentropy(W,C)
print("tree",tree)
print("HuffmanCoding",C)
print(r,l)
```

```
tree [0.9986, [0.4046, [0.1882, [0.0931, 'p'], [0.0951, 'f']], [0.2164, [0.1051, [0.0522, 'v'],
[0.0529, 'd']], [0.1113, [0.0533, [0.0265, 'u'], [0.0268, 'e']], [0.058, [0.0274, 'g'], [0.0306,
'w']]]], [0.594, [0.266, [0.125, [0.0587, 't'], [0.0663, 'c']], [0.141, [0.0674, 'q'], [0.0736, 'k']],
[0.328, [0.1567, [0.0764, 'l'], [0.0803, 'x']], [0.1713, [0.0829, [0.0368, [0.0172, [0.0084,
[0.0034, [0.0012, 'b'], [0.0022, 'r']], [0.005, 'y']], [0.0088, 'n']], [0.0196, [0.0097, 'h'], [0.0099,
'a']], [0.0461, [0.0205, 'j'], [0.0256, [0.0125, 'z'], [0.0131, [0.006, [0.0029, 'i'], [0.0031, 'm']],
[0.0071, 'o']]]]]], [0.0884, 's']]]]]
HuffmanCoding [['p', '000'], ['f', '001'], ['v', '0100'], ['d', '0101'], ['u', '01100'], ['e', '01101'], ['g',
'01110'], ['w', '01111'], ['t', '1000'], ['c', '1001'], ['q', '1010'], ['k', '1011'], ['l', '1100'], ['x', '1101'],
['b', '111000000'], ['r', '111000001'], ['y', '11100001'], ['n', '1110001'], ['h', '1110010'], ['a',
'1110011'], ['j', '111010'], ['z', '1110110'], ['i', '111011100'], ['m', '111011101'], ['o', '11101111'],
['s', '1111']]
4.142 4.176
```



符号的频率、编码与码长

- 熵= $\sum_k p_k \log_2(p_k) = 4.173$
- 统计英文书，完成词频表
- 概率平均码长= $\sum_k p_k m_k = 4.2403$
- 编码、解码

a	0.0817	1101'	4	0.3268
b	0.0145	1100'	4	0.058
c	0.0248	000000'	6	0.1488
d	0.0431	00001'	5	0.2155
e	0.1232	100'	3	0.3696
f	0.0209	000101'	6	0.1254
g	0.0182	011000'	6	0.1092
h	0.0668	0100'	4	0.2672
i	0.0689	0011'	4	0.2756
j	0.001	0000010101'	10	0.01
k	0.008	00000100'	8	0.064
l	0.0397	00011'	5	0.1985
m	0.0277	10110'	5	0.1385

n	0.0662	0101'	4	0.2648
o	0.0781	0010'	4	0.3124
p	0.0156	011001'	6	0.0936
q	0.0009	00000101000'	11	0.0099
r	0.0572	1010'	4	0.2288
s	0.0628	0111'	4	0.2512
t	0.0905	111'	3	0.2715
u	0.0304	01101'	5	0.152
v	0.0102	0000011'	7	0.0714
w	0.0264	10111'	5	0.132
x	0.0015	000001011'	9	0.0135
y	0.0211	000100'	6	0.1266
z	0.0006	00000101001'	11	0.0055



一位老木匠需要将一根长的木棒切成N段。每段的长度分别为 $L_1, L_2, \dots, L_N$ ($1 \leq L_1, L_2, \dots, L_N \leq 1000$ ，且均为整数)个长度单位。我们认为切割时仅在整数点处切且没有木材损失。

木匠发现，每一次切割花费的体力与该木棒的长度成正比，不妨设切割长度为1的木棒花费1单位体力。例如：若 $N=3$ ， $L_1=3, L_2=4, L_3=5$ ，则木棒原长为12，木匠可以有多种切法，如：先将12切成3+9，花费12体力，再将9切成4+5，花费9体力，一共花费21体力；还可以先将12切成4+8，花费12体力，再将8切成3+5，花费8体力，一共花费20体力。显然，后者比前者更省体力。

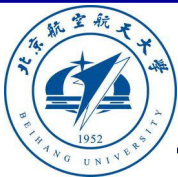
那么，木匠至少要花费多少体力才能完成切割任务呢？

直接应用霍夫曼编码！为什么？



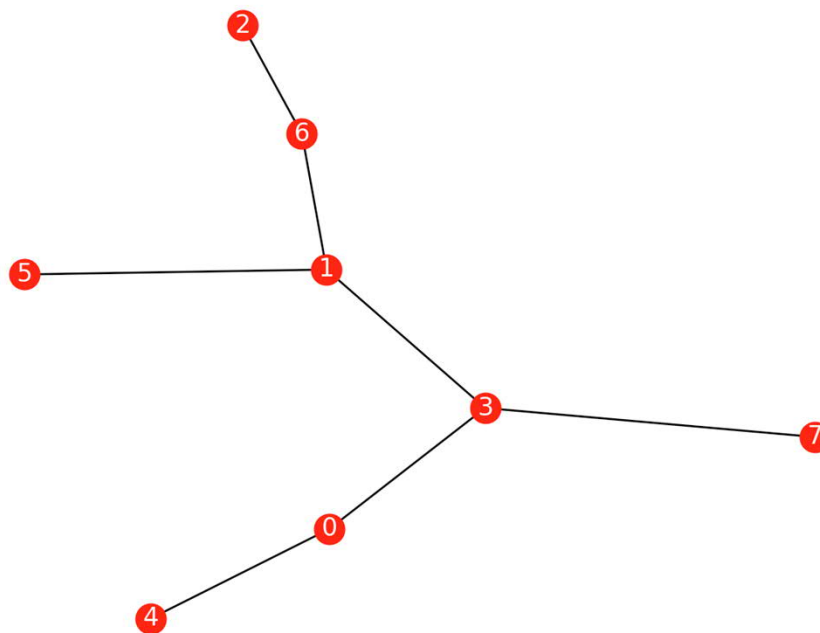
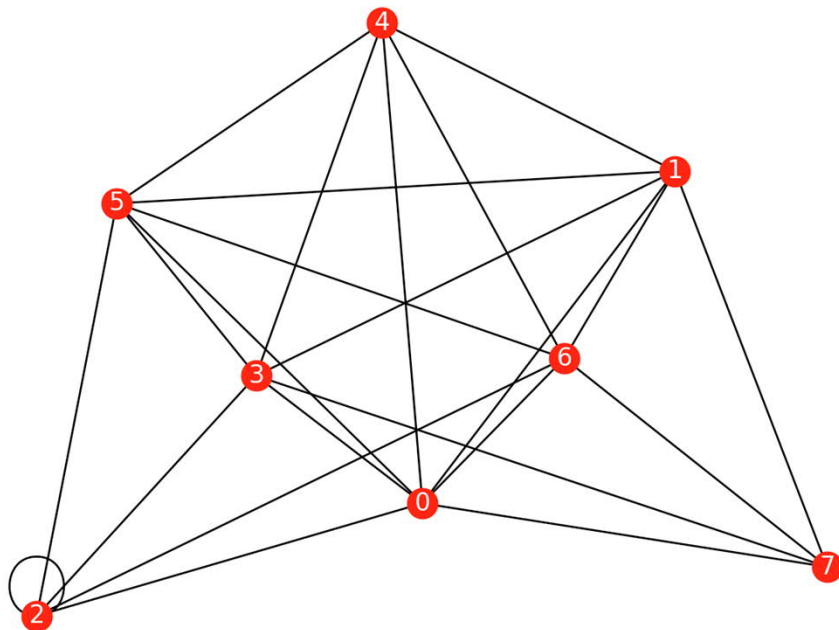
本章内容

- 树的概念
 - 定义与判断
 - 根树与有序树
- 二叉树及遍历
 - 二叉树的表示
 - 二叉树的遍历
- 霍夫曼编码
 - 霍夫曼树构建
 - 霍夫曼编码
- 最小生成树
 - 最小生成树的定义
 - 普里姆算法
 - 克鲁斯卡尔算法



生成树

- 设图 $G = \langle V, E \rangle$, 从图 G 能够生成一个图 $G_T = \langle V_T, E_T \rangle$, 其中 $V_T = V$ 。





生成树

- 设图 $G = \langle V, E \rangle$, 从图 G 能够生成一个图 $G_T = \langle V_T, E_T \rangle$, 其中 $V_T = V$ 。

```
import graph.graph as gg
V = {0,1,2,3,4,5,6,7}
E={ (7,3), (1,3), (1,6), (3,7), (2,5), (0,3), (5,1), (4,0), (6,7), (5,0), (0,4), (2,2), (6,4), (5,4), (2,6), (3,2), (7,1), (6,0), (1,4), (2,3), (1,0), (6,5), (5,3), (0,1), (3,5), (7,0), (4,6), (4,3), (3,5), (0,2) }
V = {0,1,2,3,4,5,6,7}
Et={ (7,3), (1,3), (1,6), (2,6), (5,1), (0,3), (4,0) }
gg.drawgraph(E)
gg.drawgraph(Et)
tv = len(V) == len(Vt)
print(tv)
```

True



生成树

- 从图 G 构建树 G_T 的算法思路如下
 - 1) 构造图 $G(m, n)$;
 - 2) 在边集合任选一条边 E_T ;
 - 3) 在生成树中 G_T , 如果 $len(G_T) < n - 1$, 则拓展树 G_T ;
 - 4) 若 G 连通子图有新边, 即, 对于任意 $(u, v) \in E$, 如果 $u \in V_T$ 且 $v \notin V_T$, 或者, $u \notin V_T$ 且 $v \in V_T$, 则 $(u, v) \in E_T$;
 - 5) 若 G 连通子图无新边, 即 $r = len(V_T)$, 则终止循环, 即终止扩展 G_T ;
 - 6) 重复 (3) $\sim$ (5) ;



生成树

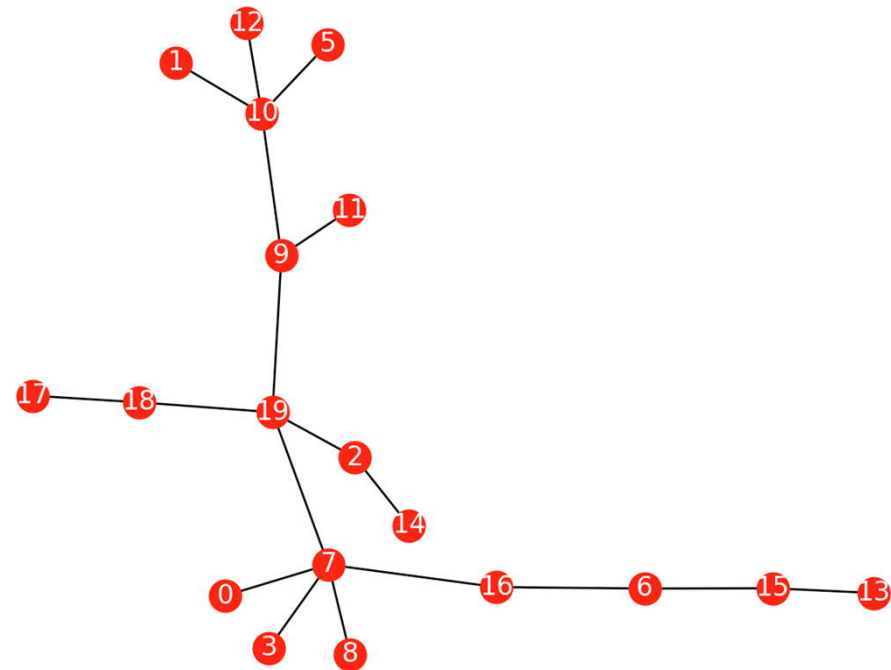
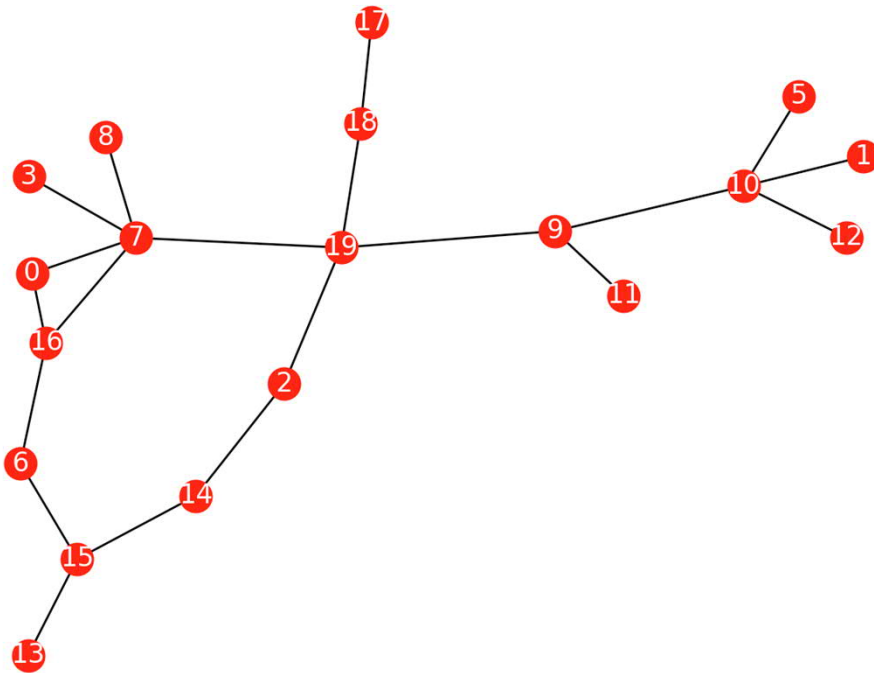
```
def graph2tree(V, E):  
    E = sorted(E)  
    (u, v) = E[0]  
    Vt = {u, v}  
    Et = {(u, v)}  
    E = set(E)  
    n = len(V)  
    while (len(Et) < (n - 1)):  
        m = len(Vt)  
        for (u, v) in E:  
            if ((u in Vt and v not in Vt)  
                or (u not in Vt and v in Vt)):  
                Vt = Vt | {u, v}  
                Et = Et | {(u, v)}  
        E = E - Et  
        if (m == len(Vt)):  
            break  
    return [Vt, Et]
```



生成树

```
import graph.graph as gg
m,n=20,20
[V,E] = gg.creategraph(m,n)
[Vt,Et] = gg.graph2tree(V,E)
gg.drawgraph(E)
gg.drawgraph(Et)
print(Et)
```

```
{(0, 7), (10, 8), (10, 14), (3, 1), (10, 17), (5, 1), (16, 12), (17, 6), (9, 5), (3, 2), (4, 1), (10, 18), (0, 11), (1, 12), (6, 2), (13, 12), (11, 18)}
```





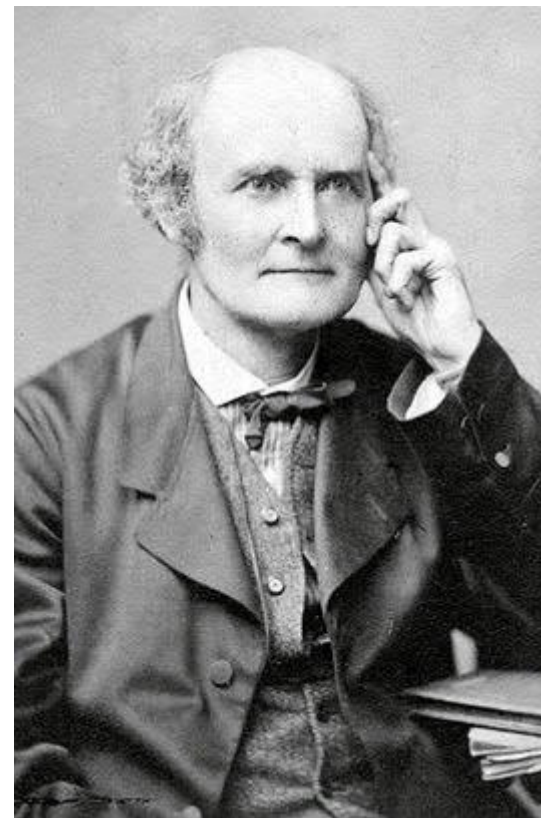
生成树定理

- **定理 12.5.1** 设 $G = \langle V, E \rangle$ 是无向图，并且 G 是连通的，则 G 至少有一个生成树。
- 证明：
 - 因为图 G 是连通的，所以有 $\forall u \forall v (u \in V \wedge v \in V \rightarrow u \Rightarrow v)$
 - 若 $(u, v) \in E$ ，则 $E' = E - \{(u, v)\}$ ，即 $\forall u (u \in V \wedge (u, v) \in E \rightarrow E' = E - \{(u, v)\})$ ，保持 $G' = \langle V, E' \rangle$ 是连通的。
 - 若 $u \Rightarrow v$ ，则删除环上一条边，不妨设 $(u, v) \in E$ ， $E' = E - \{(u, v)\}$ $\forall u (u \in T \rightarrow \neg(u \Rightarrow v))$ ，保持 $G = \langle V, E' \rangle$ 连通
 - 所以， $G = \langle V, E' \rangle$ 连通、无环且节点集不变，即 $\forall u \forall v (u \in T \wedge v \in T \rightarrow u \Rightarrow v) \wedge \forall u (u \in T \rightarrow \neg(u \Rightarrow u))$ ， G 至少有一颗生成树。



生成树的数量

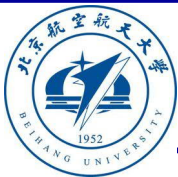
- 阿瑟·凯莱 (Arthur Cayley)
- 定理：设 $G = \langle V, E \rangle$ ， n 阶图的生成树数有 n^{n-2} 。



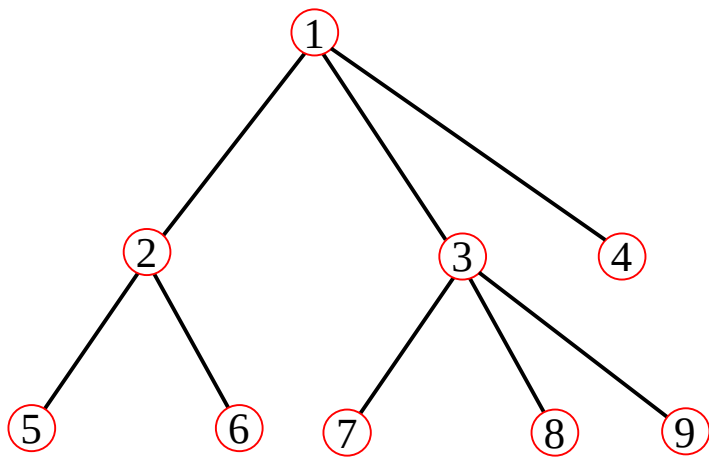


生成树的数量

- 定理：设 $G = \langle V, E \rangle$ ， n 阶图的生成树数有 n^{n-2} 。
- 建立一棵 n 阶树和 $(m_1, m_2, \dots, m_{n-2})$ 的 $n - 2$ 维向量建立一一对应关系；其中 $m_i \in [1, n]$
- 可以使用prufer编码。
- 如果能建立这样的联系，所有可能的向量为 n^{n-2} 。因此，生成树数有 n^{n-2}

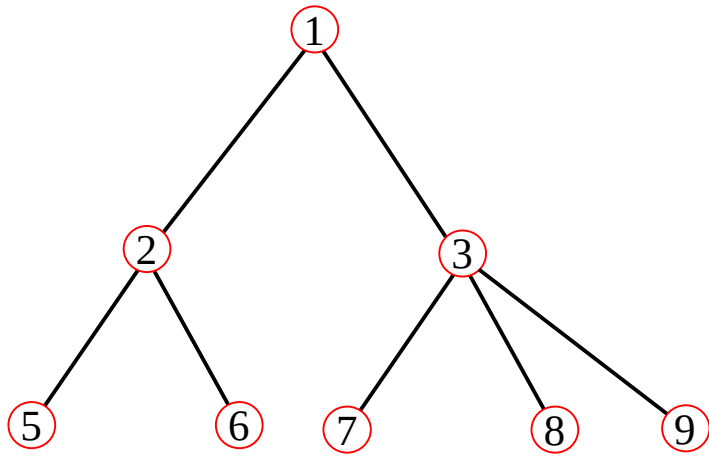


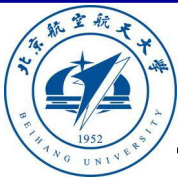
Prufer编码



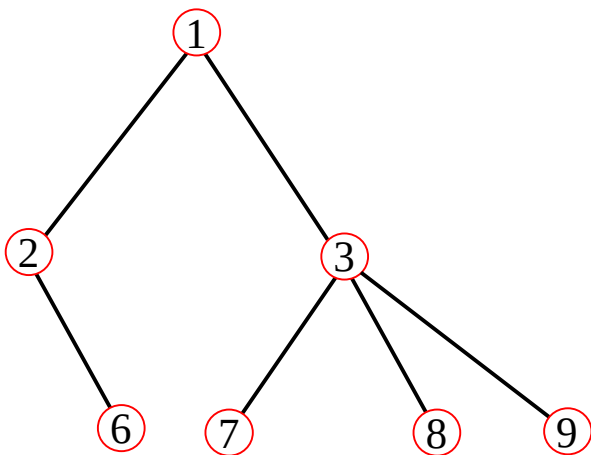


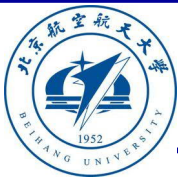
Prufer编码



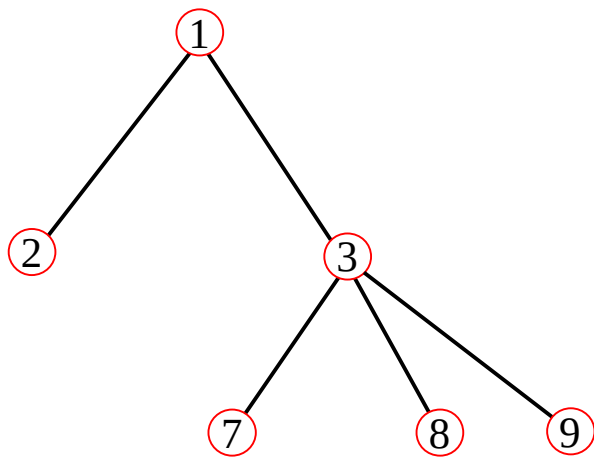


Prufer编码

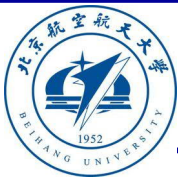




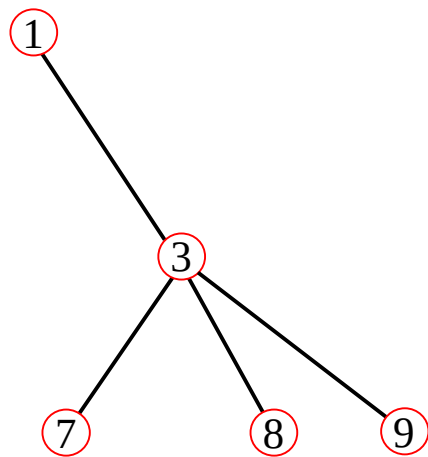
Prufer编码



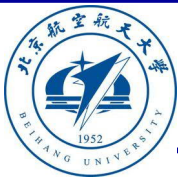
1	2	2				
---	---	---	--	--	--	--



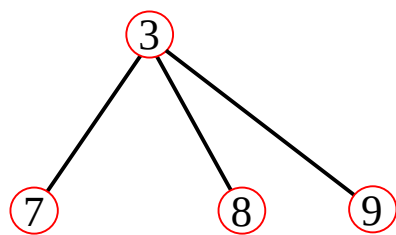
Prufer编码



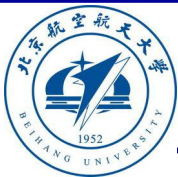
1	2	2	1			
---	---	---	---	--	--	--



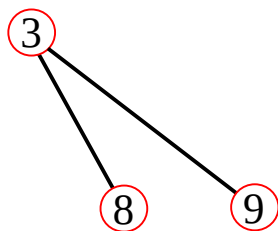
Prufer编码



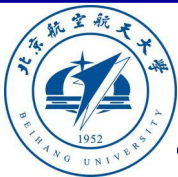
1	2	2	1	3		
---	---	---	---	---	--	--



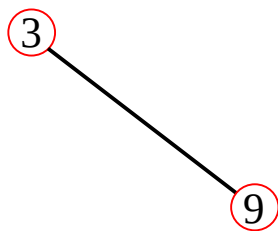
Prufer编码



1	2	2	1	3	3	
---	---	---	---	---	---	--



Prufer编码



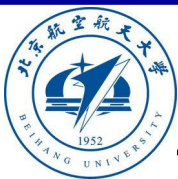
1	2	2	1	3	3	3
---	---	---	---	---	---	---



Prufer编码

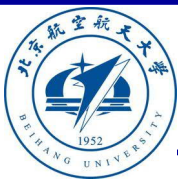
1	2	3	4	5	6	7	8	9
3	3	4	1	1	1	1	1	1

1	2	2	1	3	3	3
---	---	---	---	---	---	---

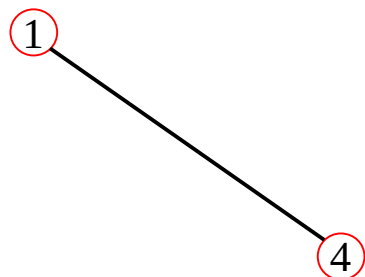


Prufer编码

1	2	3	4	5	6	7	8	9
3	3	4	1	1	1	1	1	1
			↑					
1	2	2	1	3	3	3		
↑								

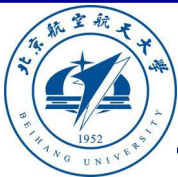


Prufer编码

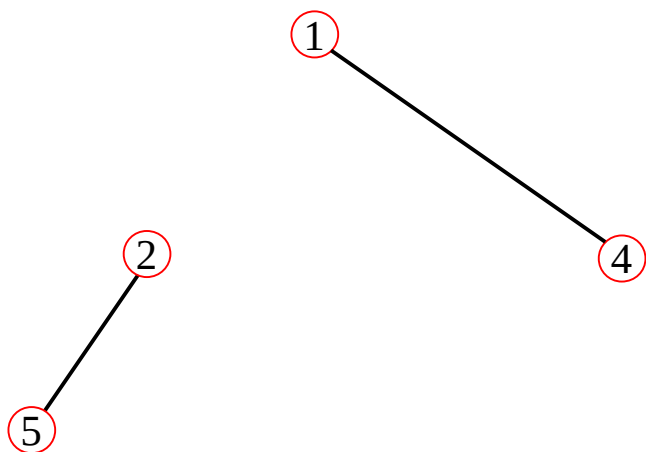


1	2	3	4	5	6	7	8	9
2	3	4	0	1	1	1	1	1
1	2	2	1	3	3	3		

Red arrows point to the '1' in the 5th column of the second row and the '2' in the 2nd column of the third row.



Prufer编码

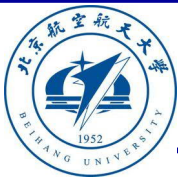


1	2	3	4	5	6	7	8	9
2	2	4	0	0	1	1	1	1

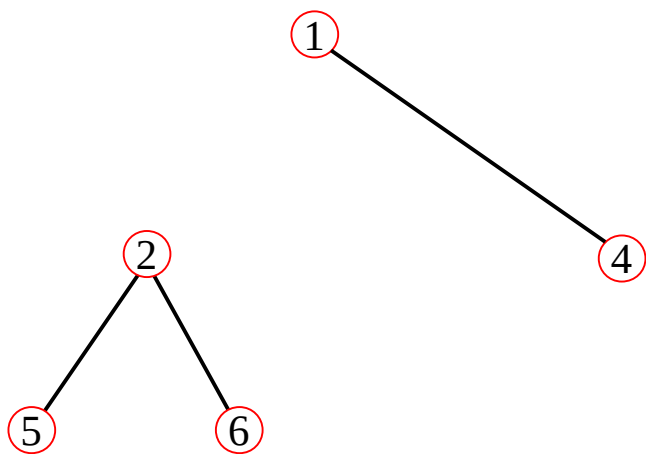


1	2	2	1	3	3	3
---	---	---	---	---	---	---



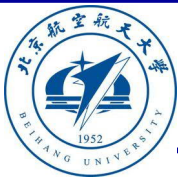


Prufer编码

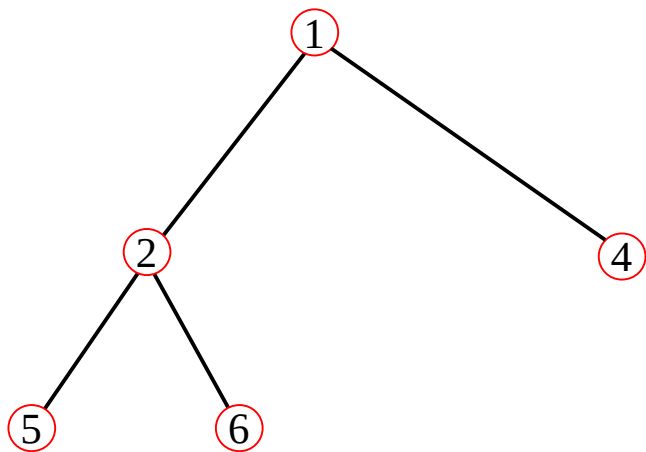


1	2	3	4	5	6	7	8	9
2	1	4	0	0	0	1	1	1
1	2	2	1	3	3	3		

Red arrows point to the value 1 in the second row, first column and the value 1 in the third row, fourth column.

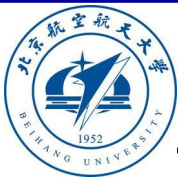


Prufer编码

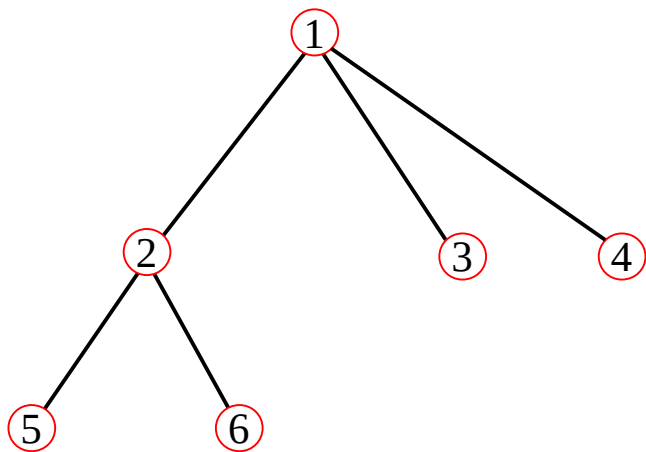


1	2	3	4	5	6	7	8	9
1	0	4	0	0	0	1	1	1
1	2	2	1	3	3	3		





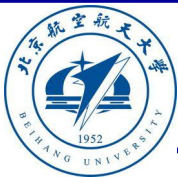
Prufer编码



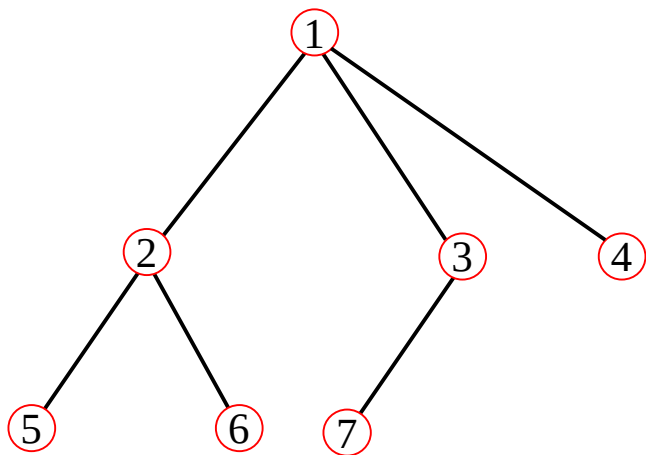
1	2	3	4	5	6	7	8	9
0	0	3	0	0	0	1	1	1

1	2	2	1	3	3	3
---	---	---	---	---	---	---





Prufer编码

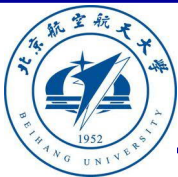


1	2	3	4	5	6	7	8	9
0	0	2	0	0	0	0	1	1

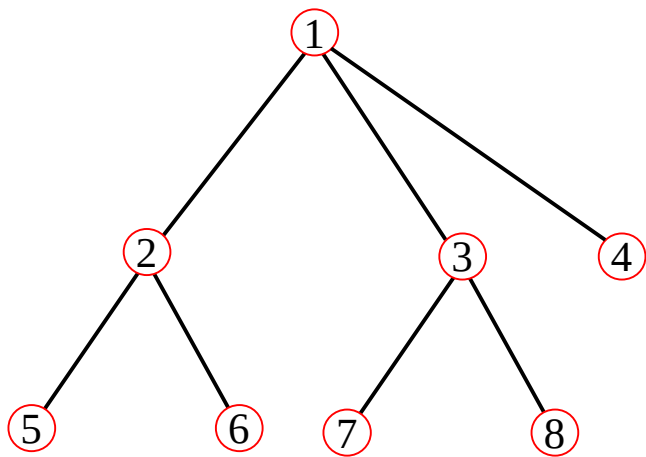


1	2	2	1	3	3	3
---	---	---	---	---	---	---

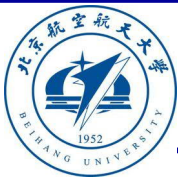




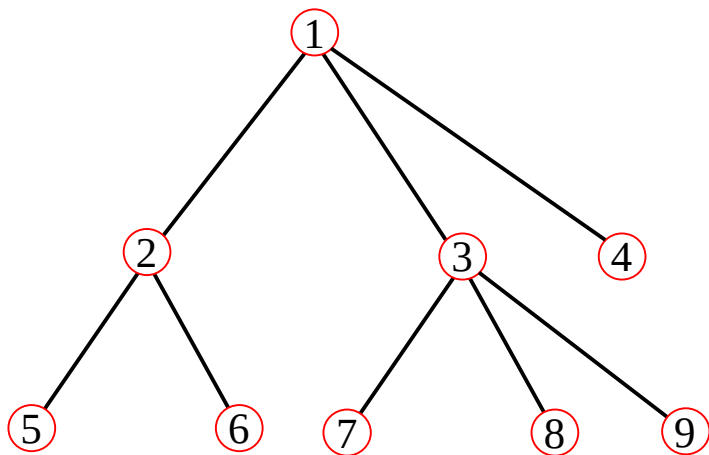
Prufer编码



1	2	3	4	5	6	7	8	9
0	0	1	0	0	0	0	0	1
1	2	2	1	3	3	3		



Prufer编码



1	2	3	4	5	6	7	8	9
0	0	0	0	0	0	0	0	0

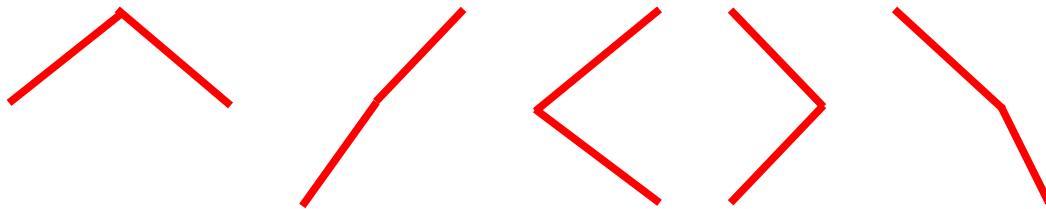
1	2	2	1	3	3	3
---	---	---	---	---	---	---



二叉树

- n 个节点构成的二叉树，共有 $\frac{1}{n+1} C_{2n}^n$ 种。
- 递推式子：
- $h(n) = h(0)h(n-1) + \dots + h(n-1)h(0)$
- $h(0) = 1, h(1) = 1, h(2) = 2, h(3) = 5$

定义 12.2.3 设 V 是非空节点集合， $E = \{\langle u, v, k \rangle | u \in V \wedge v \in V, k = 1, 2, \dots, m\}$ ， $T = \langle V, E \rangle$ 是树，并且每个节点的最多有 m 个子节点，则称为 m 元树。



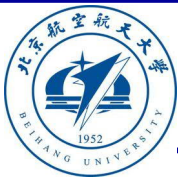


$m*n$ 阵列图

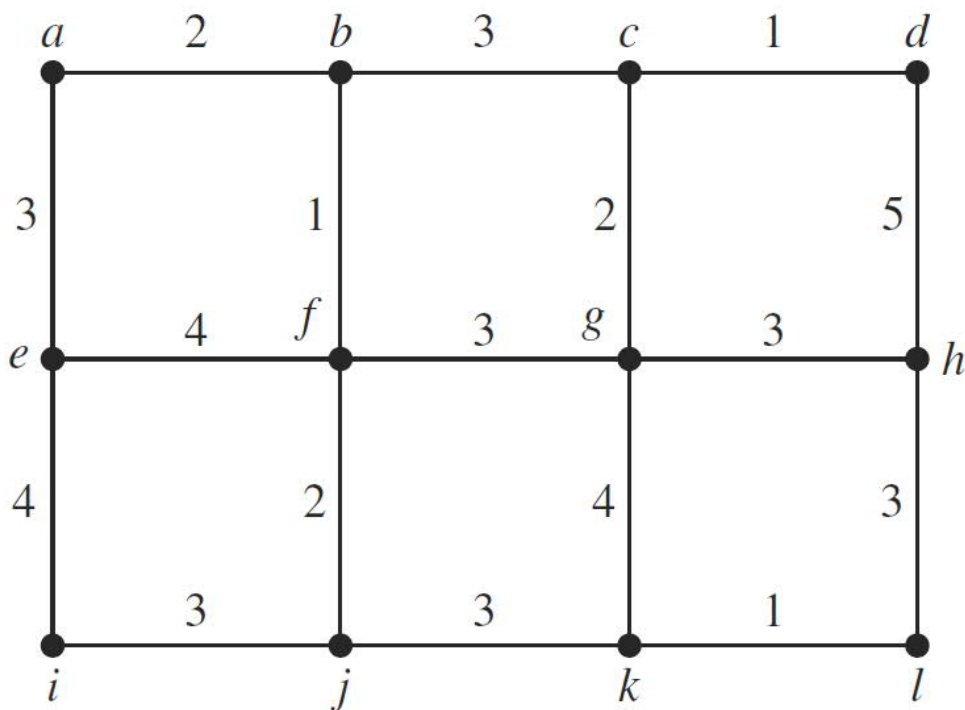
```
def MNgraph(m,n):
    V=set({})
    E=set({})
    for i in range(m):
        for j in range(n-1):
            u=i*n+j
            v=u+1
            V=V|{u,v}
            E=E|{(u,v)}
    for i in range(m-1):
        for j in range(n):
            u=i*n+j
            v=u+n
            E=E|{(u,v)}
    return [V,E]
```

```
def weightedgraph(V,E,W):
    V=V
    E=set({})
    for (u,v) in E:
        w=random.randint(1, W)
        E=E|{(w,u,v)}
    return [V,E]
```

```
import graph.graph as gg
m,n,W = 5,4,10
[V,E] = gg.MNgraph(m,n)
gg.drawgraph(E)
[Vt,Et] =
gg.weightedgraph(V,E,W)
print(E)
```



m*n阵列图



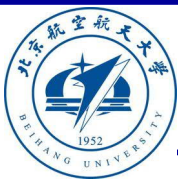
(a,b)	2
(b,c)	3
(c,d)	1
(a,e)	3
(b,f)	1
(c,g)	2
(d,h)	5
(e,f)	4
(f,g)	3
(g,h)	3
(e,i)	4
(f,j)	2
(g,k)	4
(h,l)	3
(i,j)	3
(j,k)	3
(k,l)	1

(c,d)	1
(b,f)	1
(k,l)	1
(a,b)	2
(c,g)	2
(f,j)	2
(b,c)	3
(a,e)	3
(f,g)	3
(g,h)	3
(h,l)	3
(i,j)	3
(j,k)	3
(e,f)	4
(e,i)	4
(g,k)	4
(d,h)	5



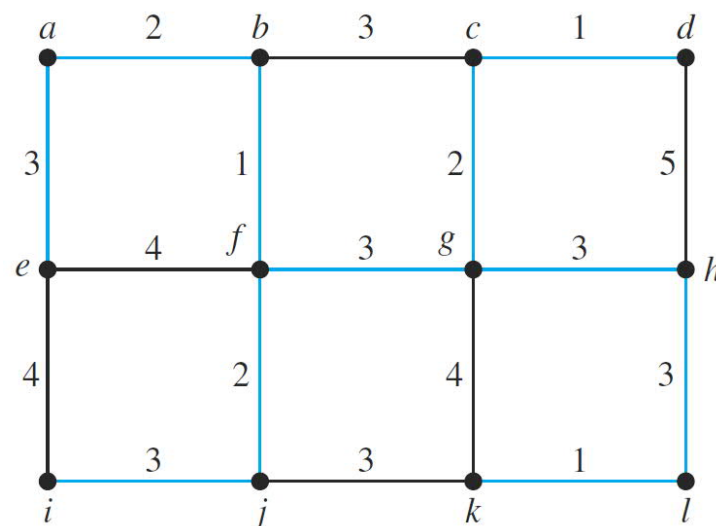
最小生成树

- **定义 12.5.2** 设 $G = \langle V, E \rangle$ 是无向图, T 是 G 的生成树, 并且权之和最小, 称为 T 为 G 的最小生成树。
- **定义 12.5.3** 设 T 是带权连通无向图 G 的生成树, T 的各边权值之和称为 T 的权, 记为 $C(T)$ 。
- **定义 12.5.4** 在带权连通无向图 G 的所有生成树中, 权最小的生成树称为 G 的最小生成树。

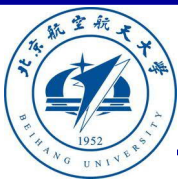


普里姆 (Prim) 算法

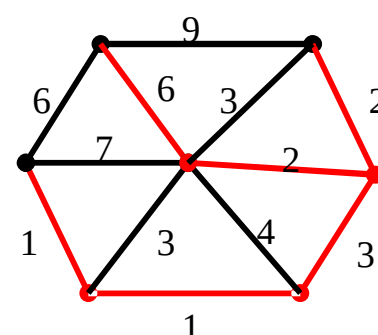
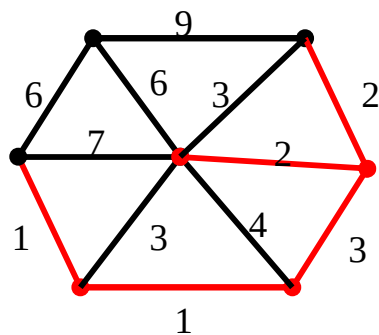
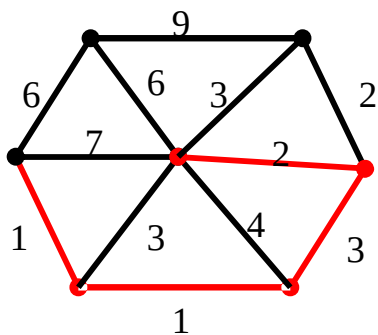
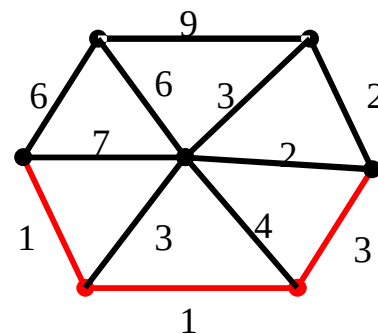
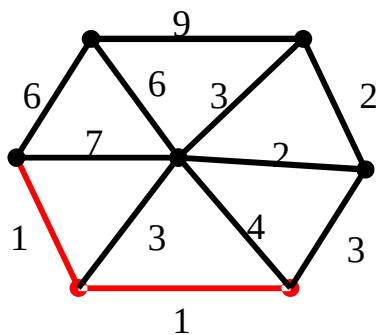
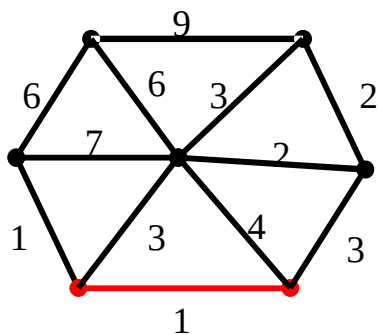
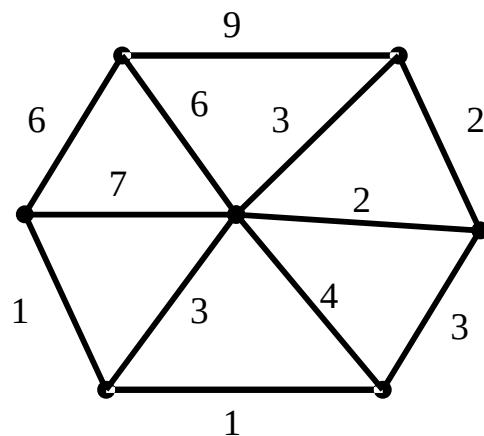
- 于1957年由罗伯特·普里姆提出
- 保持连通性的最小性避圈扩展
 - 图 $G = \langle V, E \rangle$, 树 $Gt = \langle Vt, Et \rangle$
 - 选择最小权边 (w, u, v)
 - 如果 (w, u, v) 与树 Et 节点关联, 并且在树 Et 中不形成简单回路, 则边 (w, u, v) 为树 Et 的边
 - 如果树 Et 中边的数量小于 $n - 1$, 则继续步骤1), 否则, 终止



1	('b','f'),	1
2	('a','b'),	2
3	('f','j'),	2
4	('a','e'),	3
5	('i','j'),	3
6	('f','g'),	3
7	('c','g'),	2
8	('c','d'),	1
9	('g','h'),	3
10	('h','l'),	3
11	('k','l')	1



普里姆算法





普里姆 (Prim) 算法

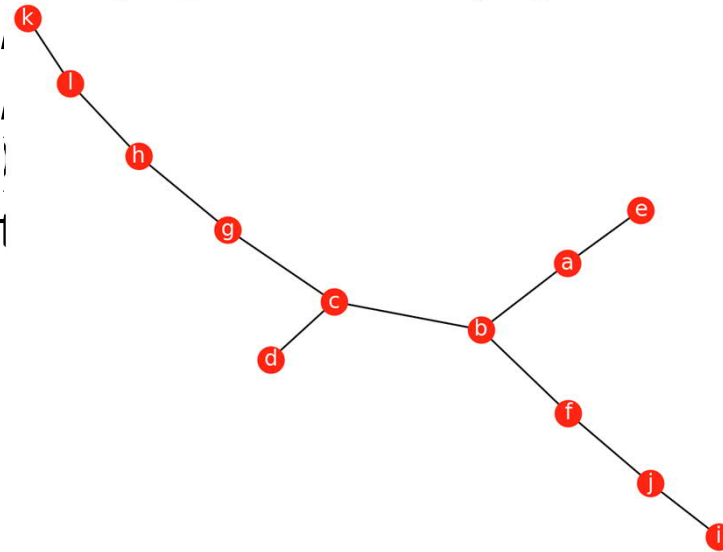
- 权边排序
- 初始化树
- n 是节点数
- 保持连通条件,
增加权值最小的边
及其节点

```
def Primspantree(V,E):  
    E=sorted(E)  
    [w,u,v]=E[0]  
    Vt={u,v}  
    Et={(w,u,v)}  
    del E[0]  
    n=len(V)  
    while (len(Et)<(n-1)):  
        i=0  
        while(i < len(E)):  
            [w,u,v]=E[i]  
            if (u in Vt and v not in Vt)or(u not in Vt and v in Vt):  
                Et=Et|{(w,u,v)}  
                Vt=Vt | {u,v}  
                del E[i]  
                break  
            i=i+1  
    return [Vt,Et]
```



普里姆 (Prim) 算法

```
import graph.graph as gg
V=['a','b','c','d','e','f','g','h','i','j','k','l']
E=[[2,'a','b'],[3,'b','c'],[1,'c','d'],[3,'a','e'],[1,'b',
'f'],[2,'c','g'],[5,'d','h'],[4,'e','f'],[3,'f','g'],[
3,'g','h'],[4,'e','i'],[2,'f',
'],[3,'i','j'],[3,'j','k'],[1,
[Vt,Et] = gg.Primspantree(V,E)
[Vt,Eg] = gg.graphw2graph(V,E)
gg.drawgraph(Eg)
print(E)
print(Et)
```

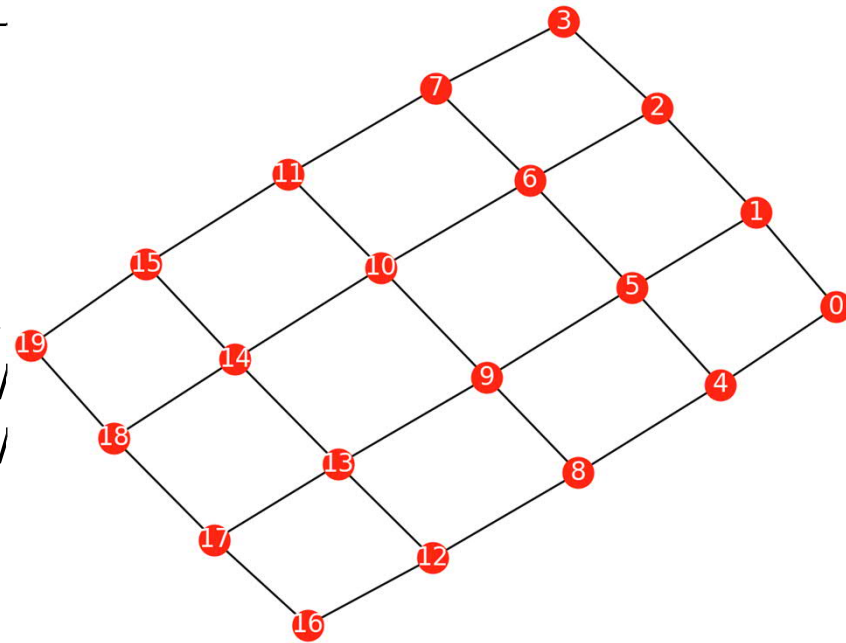


```
[[2, 'a', 'b'], [3, 'b', 'c'], [1, 'c', 'd'], [3, 'a', 'e'], [1, 'b', 'f'], [2, 'c', 'g'], [5, 'd', 'h'], [4, 'e', 'f'],
[3, 'f', 'g'], [3, 'g', 'h'], [4, 'e', 'i'], [2, 'f', 'j'], [4, 'g', 'k'], [3, 'h', 'l'], [3, 'i', 'j'], [3, 'j', 'k'], [1,
'k', 'l']]
{(2, 'f', 'j'), (3, 'h', 'l'), (1, 'k', 'l'), (2, 'c', 'g'), (3, 'i', 'j'), (3, 'a', 'e'), (3, 'g', 'h'), (2, 'a', 'b'),
(1, 'b', 'f'), (1, 'c', 'd'), (3, 'b', 'c')}
```



普里姆算法

```
import graph.graph as gg
m,n,W = 5,4,100
[V,E] = gg.MNgraph(m,n)
gg.drawgraph(E)
[V,E] = gg.weightedgraph(V,
[Vt,Et] = gg.Primspantree(V
[Vt,Eg] = gg.graphw2graph(V
gg.drawgraph(Eg)
print(E)
print(Et)
```

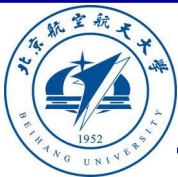


```
{(80, 13, 14), (81, 10, 14), (6, 15, 19), (54, 10, 11), (71, 17, 18), (49, 1, 2), (50,
5, 6), (25, 8, 9), (18, 0, 1), (100, 12, 16), (23, 12, 13), (63, 4, 8), (4, 6, 7), (21, 3,
7), (29, 11, 15), (75, 9, 13), (69, 16, 17), (25, 0, 4), (18, 14, 18), (33, 1, 5), (7, 2,
6), (83, 8, 12), (36, 2, 3), (41, 14, 15), (32, 4, 5), (53, 9, 10), (54, 5, 9), (10, 13,
17), (48, 7, 11), (66, 18, 19), (73, 6, 10)}
```

```
{(6, 15, 19), (71, 17, 18), (49, 1, 2), (25, 8, 9), (18, 0, 1), (23, 12, 13), (29, 11,
15), (4, 6, 7), (21, 3, 7), (69, 16, 17), (18, 14, 18), (25, 0, 4), (7, 2, 6), (41, 14,
15), (32, 4, 5), (54, 5, 9), (53, 9, 10), (10, 13, 17), (48, 7, 11)}
```

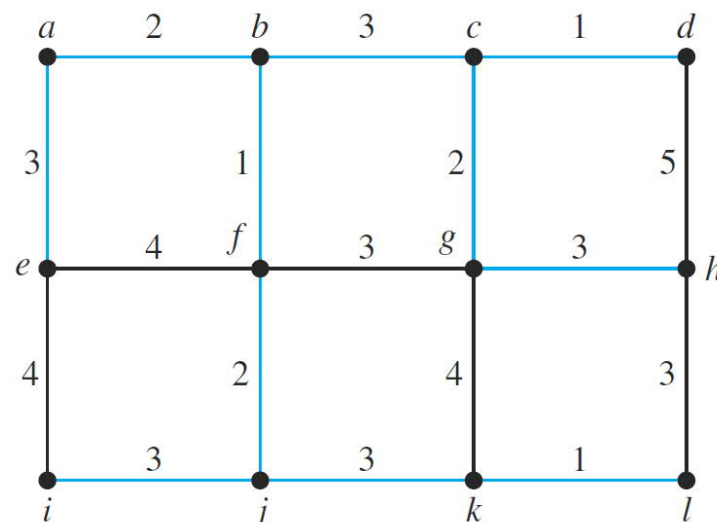


84

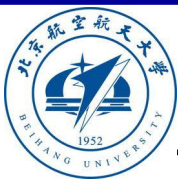


克鲁斯卡尔 (Kruskal) 算法

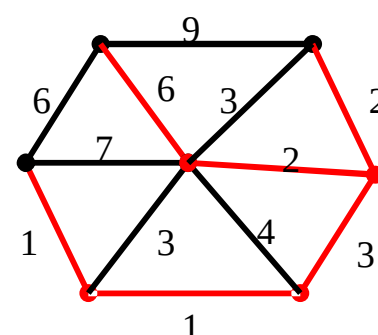
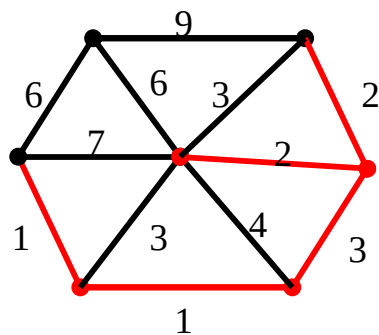
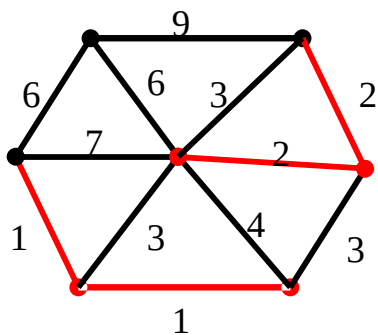
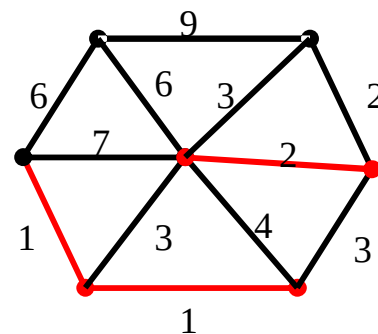
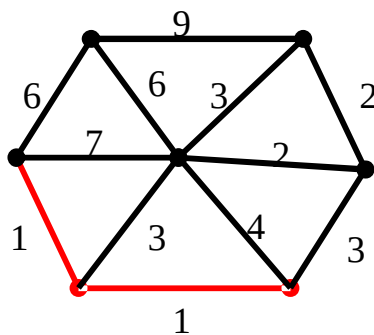
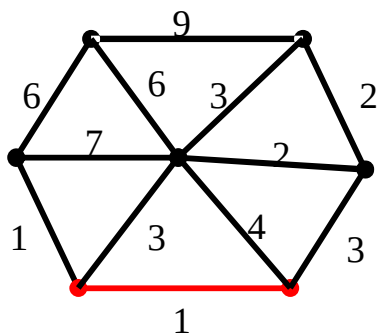
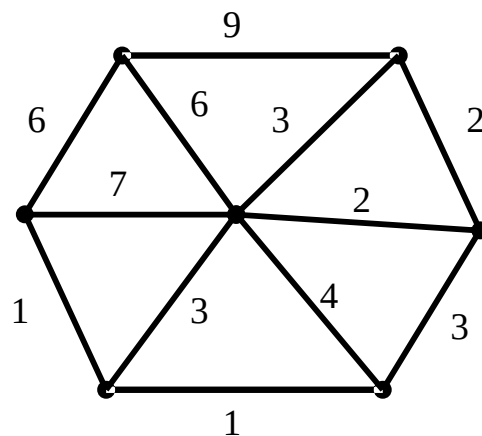
- 于1956年由约瑟夫·克鲁斯卡尔提出
- 保持最小性无圈扩展
 - 选择最小权边(w, u, v)
 - 如果(w, u, v)在树 T 中, 并且 T 不形成简单回路, 则边(w, u, v)为树 T 的边
 - 如果树 T 中边的数量不为 $n - 1$, 则继续步骤1), 否则, 终止



1	('c','d'),	1
2	('k','l'),	1
3	('b','f'),	1
4	('c','g'),	2
5	('a','b'),	2
6	('f','j'),	2
7	('b','c'),	3
8	('j','k'),	3
9	('g','h'),	3
10	('i','j'),	3
11	('a','e')	3



克鲁斯卡尔 (Kruskal) 算法





克鲁斯卡尔算法

```
def Kruskalspantree(V,E):
```

```
    E=sorted(E)
```

```
    Vt=set({})
```

```
    Et=set({})
```

```
    n=len(V)
```

```
    k=0
```

```
    i=0
```

```
    while(len(Et)<n-1):
```

```
        [w,u,v]=E[i]
```

```
        i=i+1
```

```
        if(((u in Vt) and (v in Vt))):
```

```
            if(iscycled(Vt,Et,u,v)):
```

```
                continue
```

```
            Et=Et | {(w,u,v)}
```

```
            Vt=Vt | {u,v}
```

```
    return [Vt,Et]
```

```
def iscycled(V,E,u0,v0):
```

```
    tv=False
```

```
    Vc={u0,v0}
```

```
    Ec={(u0,v0)}
```

```
    n=0
```

```
    while(n != len(Vc)):
```

```
        n=len(Vc)
```

```
        for (w,u,v) in E:
```

```
            if((u in Vc) and (v in Vc)):
```

```
                tv=True
```

```
                return tv
```

```
            if (u in Vc and v not in Vc)
```

```
or (u not in Vc and v in Vc):
```

```
                Ec=Ec | {(u,v)}
```

```
                Vc=Vc | {u,v}
```

```
                E=E-{(w,u,v)}
```

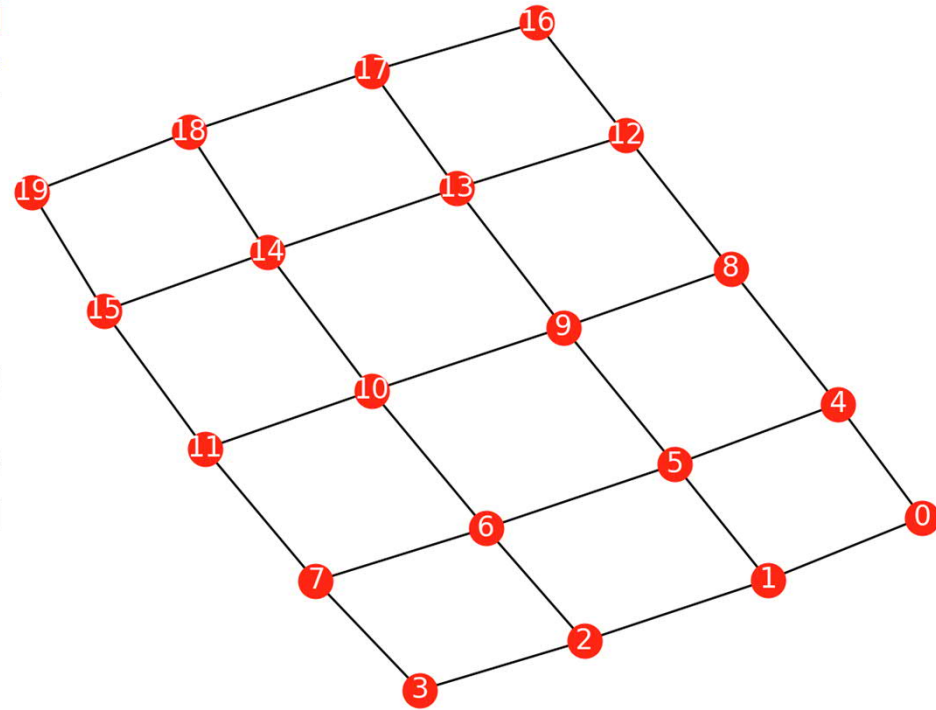
```
                break
```

```
    return tv
```



克鲁斯卡尔算法

```
import graph.graph as gg
m,n,W = 5,4,100
[V,E] = gg.MNgraph(m,n)
gg.drawgraph(E)
[V,E] = gg.weightedgraph(V,E,W)
[Vt,Et] = gg.Kruskalspantree(V,E,W)
[Vt,Eg] = gg.graphw2graph(V,Et,W)
gg.drawgraph(Eg)
print(E)
print(Et)
```



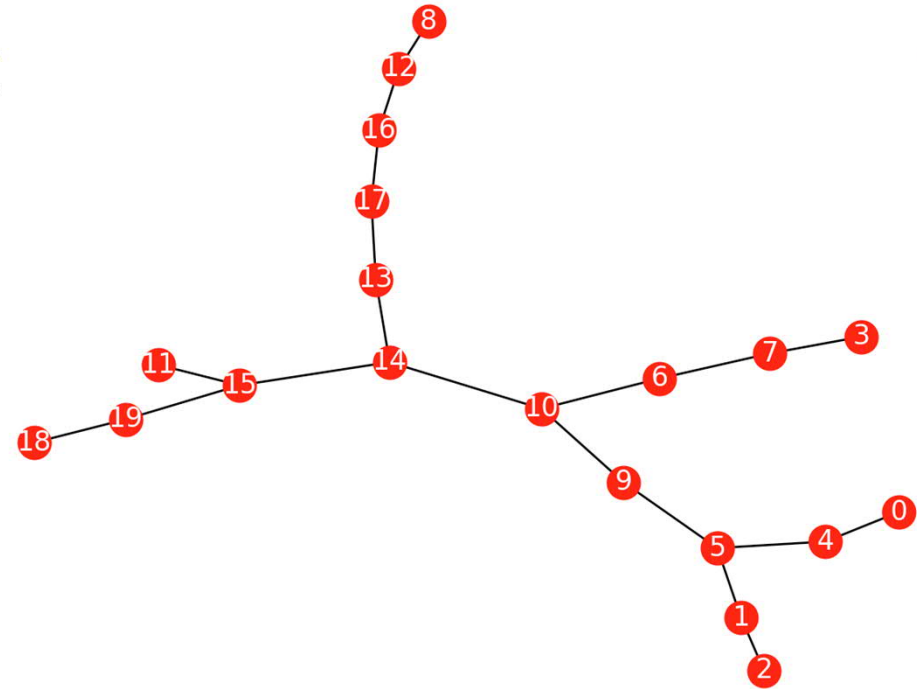
```
{(13, 10, 14), (74, 6, 10), (62, 6, 7), (95, 5, 6), (55, 5, 9), (84, 18, 19), (97, 2, 6), (57, 16, 17), (69, 15, 19), (26, 1, 5), (69, 9, 10), (81, 2, 3), (9, 13, 14), (24, 4, 5), (34, 11, 15), (74, 0, 1), (28, 12, 16), (96, 17, 18), (87, 14, 18), (96, 10, 11), (30, 0, 4), (96, 12, 13), (87, 9, 13), (72, 13, 17), (99, 7, 11), (88, 4, 8), (47, 14, 15), (42, 3, 7), (27, 8, 12), (29, 1, 2), (92, 8, 9)}
```

```
{(13, 10, 14), (74, 6, 10), (62, 6, 7), (55, 5, 9), (84, 18, 19), (57, 16, 17), (69, 15, 19), (26, 1, 5), (69, 9, 10), (9, 13, 14), (24, 4, 5), (34, 11, 15), (28, 12, 16), (30, 0, 4), (47, 14, 15), (72, 13, 17), (27, 8, 12), (42, 3, 7), (29, 1, 2)}
```



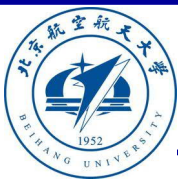
克鲁斯卡尔算法

```
import graph.graph as gg
m,n,W = 5,4,100
[V,E] = gg.MNgraph(m,n)
gg.drawgraph(E)
[V,E] = gg.weightedgraph(V,E,W)
[Vt,Et] = gg.Kruskalspantree(V,
[Vt,Eg] = gg.graphw2graph(V,Et)
gg.drawgraph(Eg)
print(E)
print(Et)
```

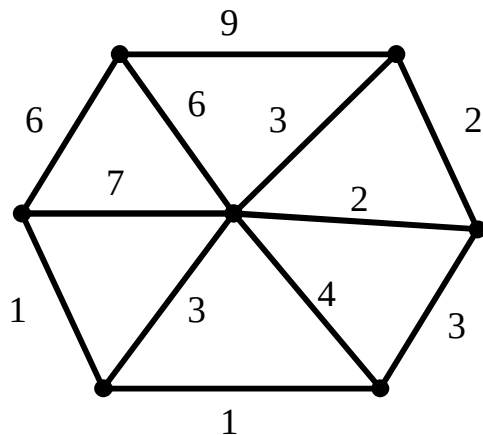


{(13, 10, 14), (74, 6, 10), (62, 6, 7), (95, 5, 6), (55, 5, 9), (84, 18, 19), (97, 2, 6), (57, 16, 17), (69, 15, 19), (26, 1, 5), (69, 9, 10), (81, 2, 3), (9, 13, 14), (24, 4, 5), (34, 11, 15), (74, 0, 1), (28, 12, 16), (96, 17, 18), (87, 14, 18), (96, 10, 11), (30, 0, 4), (96, 12, 13), (87, 9, 13), (72, 13, 17), (99, 7, 11), (88, 4, 8), (47, 14, 15), (42, 3, 7), (27, 8, 12), (29, 1, 2), (92, 8, 9)}

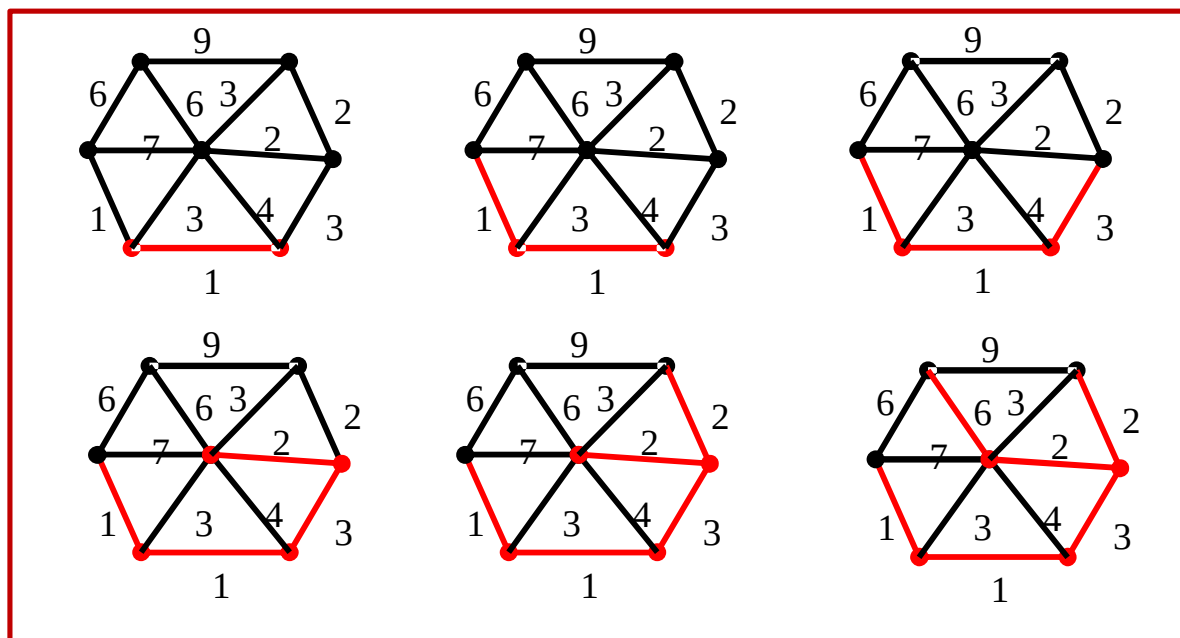
{(13, 10, 14), (74, 6, 10), (62, 6, 7), (55, 5, 9), (84, 18, 19), (57, 16, 17), (69, 15, 19), (26, 1, 5), (69, 9, 10), (9, 13, 14), (24, 4, 5), (34, 11, 15), (28, 12, 16), (30, 0, 4), (47, 14, 15), (72, 13, 17), (27, 8, 12), (42, 3, 7), (29, 1, 2)}



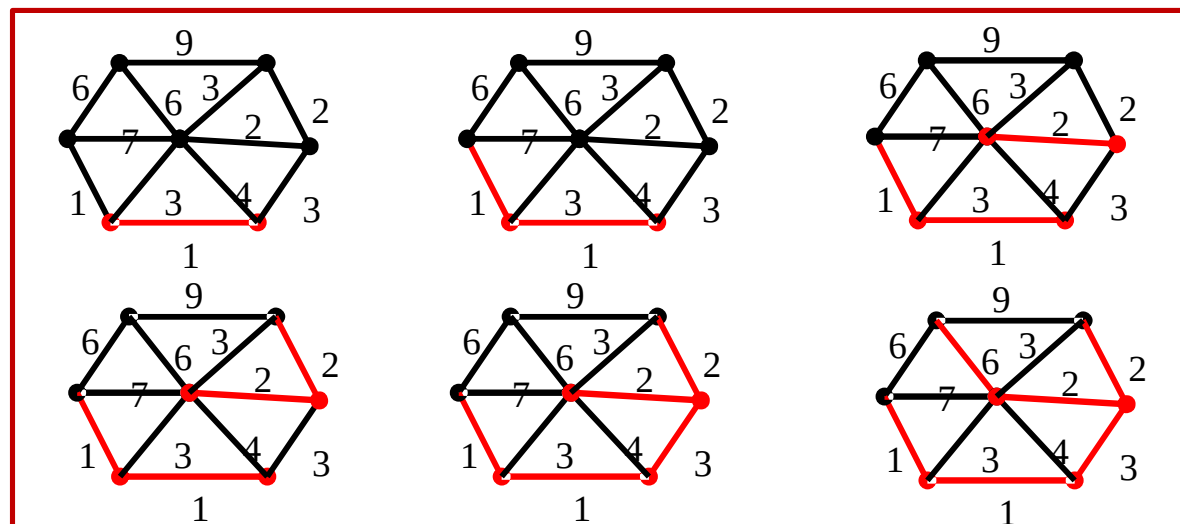
普里姆与克鲁斯卡尔算法



普里姆算法



克鲁斯卡尔算法





普里姆与克鲁斯卡尔算法（效率）

■ 复杂度

普里姆算法

$$O(|E| + |V|\log(|V|))$$

适用于稠密的图

克鲁斯卡尔算法

$$O(|E|\log(|E|)) = O(|E|\log(|V|^2)) = O(|E|\log(|V|))$$

适用于边数少的图



斯坦纳树(Steiner Tree)

- 平原上的三个城镇间要兴建一个公用的煤气供应站，在选址问题上，要考虑的主要问题是使由供应站到三个城镇的输送管道的总长最短。
- 如何去寻找合适地点？



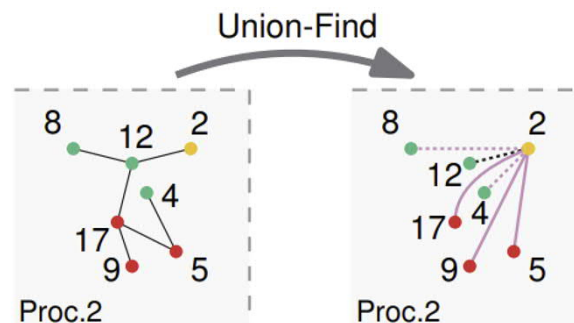
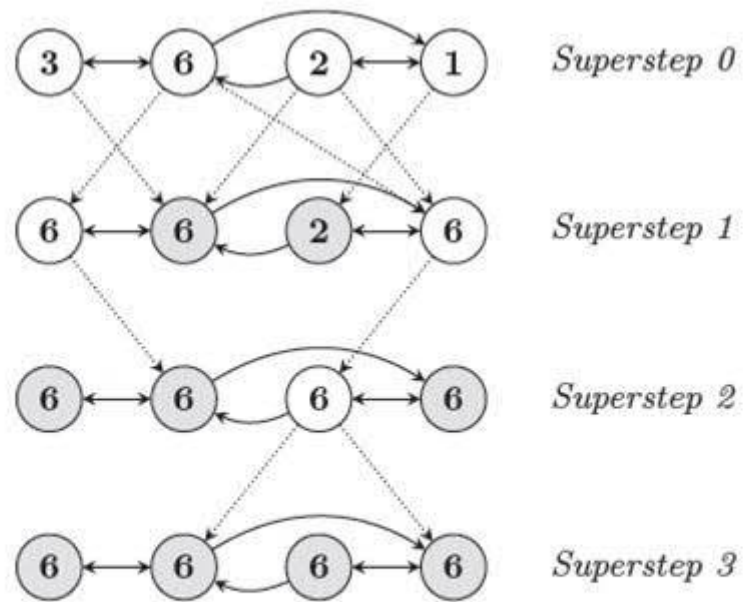
割集

- 设 G 是连通无向图， T 是 G 的一个生成树。如果从 T 中任意去掉一根树枝，分离成两个子图 T_1 和 T_2 ，子图 T_1 和 T_2 各自是连通图，但子图 T_1 和 T_2 之间相互不连通。
- 生成树 T 的每根树枝都可将 G 的节点分成两个子集，分别对应于被这根树枝分离的两个树中的节点。因此， T 的每根树枝都对应 G 的一个割集。
- **定义 12.5.5** 设 G 是连通无向图， T 是 G 的一个生成树， T 的每根树枝所对应的 G 的割集，称为对应于这根树枝的基本割集。
- **定理 12.5.2** 一个割集和任何生成树至少有一条相同边。



图计算(连通分量)

Google新 “三驾马车” —Caffeine、Pregel、Dremel





树的同态和同构匹配

- **定义10.2.10 (同构isomorphism)** : 设**无向图** $G = \langle V, E \rangle$ 和 $G' = \langle V', E' \rangle$, 如果存在**双射函数** $f: v \rightarrow v'$, 并且当且仅当 $e = (u_i, u_j) \in E$, 有 $e' = (f(u_i), f(u_j)) \in E'$, 则称 G 和 G' 同构。

设图 $G = \langle V, E \rangle$ 和 $G' = \langle V', E' \rangle$ 。

如果存在函数 $f: V \rightarrow V'$ 和 $g: E \rightarrow E'$, 使得

对于任意 $e \in E$ 及 $v_1, v_2 \in V$ 都有:

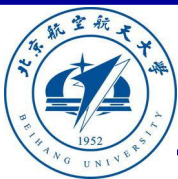
$$g(e) = \begin{cases} \{f(v_1), f(v_2)\}, & \text{若 } e = \{v_1, v_2\} \\ \langle f(v_1), f(v_2) \rangle, & \text{若 } e = \langle v_1, v_2 \rangle \end{cases}$$

则称 G 与 G' **同态**。

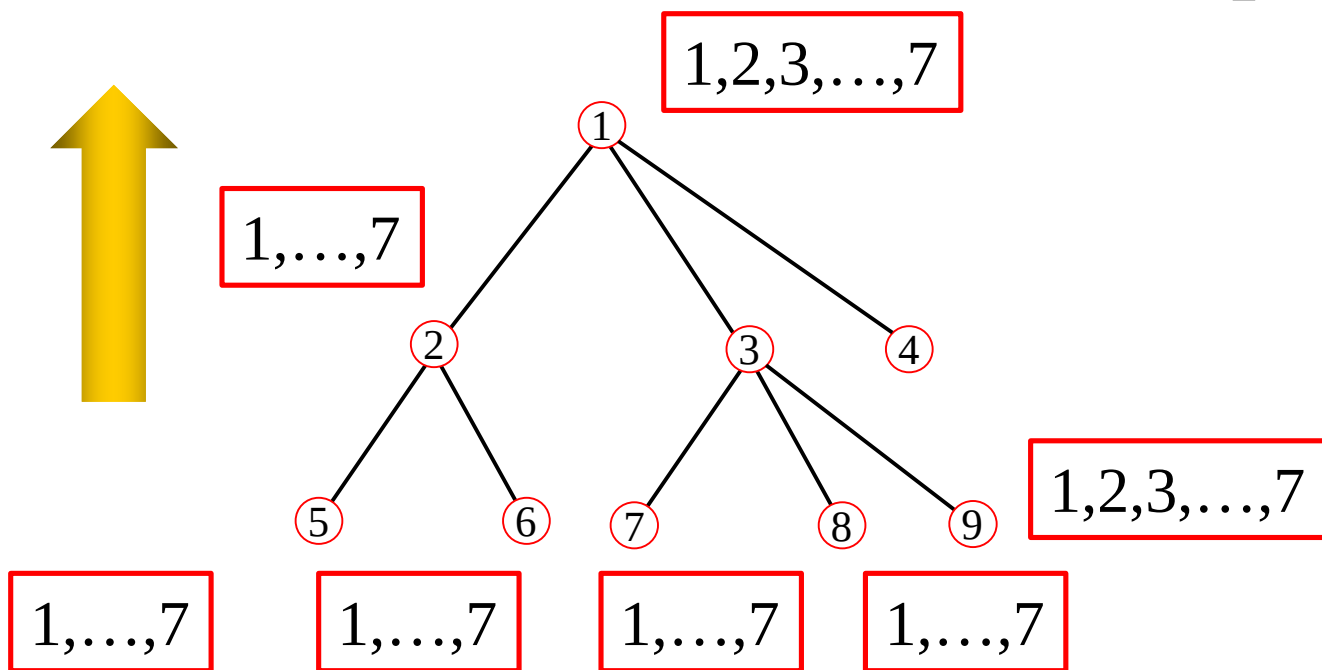
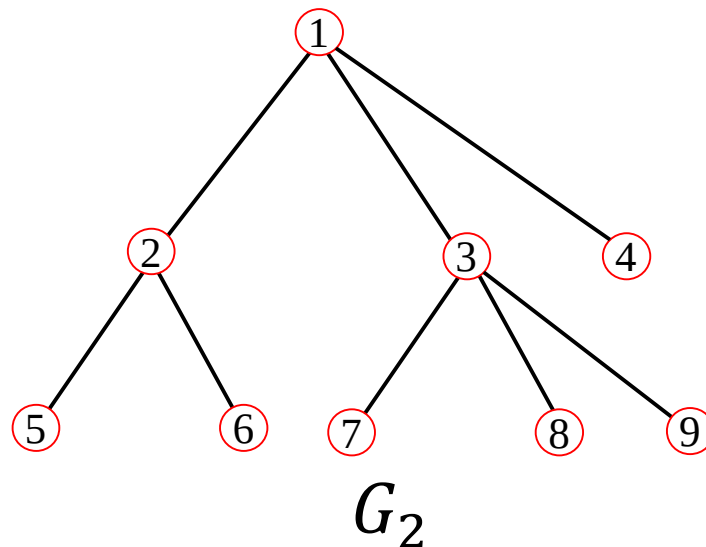
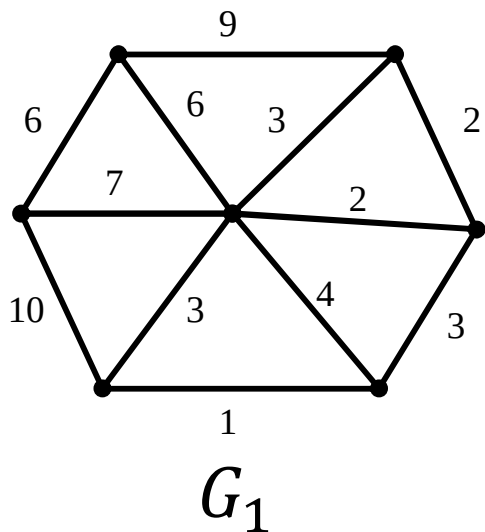


树的同态和同构匹配

给定树 T 以及图 $G = \langle V, E \rangle$, 判断 T 是否与 G 的一个子图同态?



树的同态和同构匹配



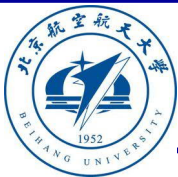


树的同态和同构匹配

给定树 T 以及图 $G = \langle V, E \rangle$ ，判断 T 是否与 G 的一个子图同态？

给定树 T 以及图 $G = \langle V, E \rangle$ ，判断 T 是否与 G 的一个子图同构？

从一个汉密尔顿路径问题构造实例(NP完全)



练习题

- 某15座城市之间的航线分属于三家航空公司. 已知无论哪一家航空公司停飞, 旅客总还能从任意城市飞往其他任何城市 (可能需要中途转机), 那么至少需要多少条航线?

如果用航线把15个城市连接, 至少需要14条.

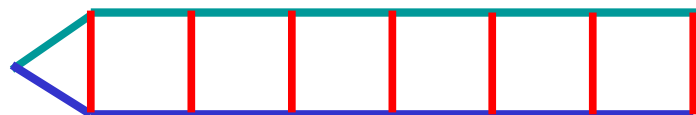
属于三家航空公司的航线数分别记为 a , b 和 c .

可知任何两家公司的航线数和都不能少于14

$$a + b \geq 14, b + c \geq 14, a + c \geq 14$$

把这三个不等式叠加后, 得 $2(a + b + c) \geq 42$

即三家航空公司航线总数不能少于21.



谢谢

